QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 1: Mathematics for AI — Intuitive Study Textbook

Linear Algebra (n×m, Im/Ker, SVD, PCA) • Matrix Calculus & Optimization • Probability, Statistics & Information Theory (M01–M20)

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

17 Sections Indexed

Day 1: Linear Algebra (Eigenvalues, Subspaces, SVD/PCA, Projections) ..... p. 2

1.1 🌟 [Core — Bread & Butter] Eigenvalues, Eigenvectors & The 15-Second Shortcut Arsenal ..... p. 2

1.2 🌟 [Core — Bread & Butter] Rank & The Four Fundamental Subspaces (Image Space $\mathrm{Im}(A)$ & Kernel $\mathrm{Ker}(A)$) ..... p. 5

1.3 🌟 [Core — Bread & Butter] Symmetric, Orthogonal, and Positive Semi-Definite (PSD) Matrices ..... p. 8

1.4 🌟 [Core — Bread & Butter] Singular Value Decomposition (SVD: $A = U\Sigma V^T$), Pseudo-Inverse & PCA ..... p. 10

1.5 🌟 [Core — Bread & Butter] Orthogonal Projections & Least Squares Geometry ..... p. 15

Day 2: Multivariable Matrix Calculus & Optimization ..... p. 17

2.1 🌟 [Core — Bread & Butter] Matrix Calculus Without Memorization ..... p. 17

2.2 🌟 [Core — Bread & Butter] Canonical Deep Learning Backprop Derivations (Softmax + CE & Linear Layers) ..... p. 18

2.3 🌟 [Core — Bread & Butter] Convexity, Gradient Descent Step-Size Limits & Newton's Method ..... p. 21

2.4 🔬 [Advanced / Deep-Dive] Constrained Optimization: Lagrange Multipliers & KKT Conditions ..... p. 24

Day 3: Probability, Statistics & Information Theory ..... p. 27

3.1 🌟 [Core — Bread & Butter] Conditional Probability, Bayes' Theorem & The Odds-Ratio Speed Hack ..... p. 27

3.2 🌟 [Core — Bread & Butter] Core Probability Distributions & Their ML Roles ..... p. 28

3.3 🌟 [Core — Bread & Butter] Expectation, Variance, Correlated Ensembles & Probability Inequalities ..... p. 30

3.4 🌟 [Core — Bread & Butter] Maximum Likelihood (MLE) vs. MAP & Estimator Bias ..... p. 33

3.5 🌟 [Core — Bread & Butter] Information Theory: Entropy, Cross-Entropy, KL Divergence & Mutual Information ..... p. 35

> ⚡ **IMPORTANT**
>
> **How This Study Guide Is Structured:** Instead of throwing dense walls of equations at you, every topic in this guide is broken down into **4 human-friendly steps**:
>
> 1. 💡 **Intuition (Plain English First):** *What problem does this solve, why does it exist, and what is the physical/visual mental model?* (With **inline visual diagrams** right where you need them!)
> 2. 📐 **The Math & Every Symbol Explained:** Clean equations where **every single letter, subscript, and matrix dimension ($n \times m$ = n rows, m columns)** is explicitly defined.
> 3. 🔢 **Step-by-Step Toy Example:** A concrete calculation with small numbers worked out line by line so you can see how the math actually works in practice.
> 4. 🎯 **Exam Takeaway & Fast Shortcuts:** The exact 60-second trick or conceptual trap tested on the Qualcomm HackerRank MCQ assessment.

> 💡 **TIP**
>
> 🎯 **80/20 Bread & Butter Priority Map (How to Study Without Getting Overwhelmed):**
>
> - 🌟 **[Core — Bread & Butter] (80% of Exam Marks):** Focus on **Trace/Determinant eigenvalue shortcuts**, **Rank-Nullity ($r + \text{nullity} = m$ columns)**, **PCA variance explained ($\sigma_i^2 / \sum \sigma_j^2$)**, **Least Squares projections ($P^2 = P$, $\text{Tr}(P) = r$)**, **Softmax + CE gradient ($p - y$)**, **GD stability ($\eta < 2/\lambda_{\max}$)**, **Bayes Odds-Ratio**, **Linearity of Expectation & Variance ($\text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y)$)**, **MLE vs. MAP ($L_2/L_1$)**, and **Entropy / Cross-Entropy / KL Divergence**.
> - 🔬 **[Optional PhD Deep-Dives] (Collapsible `<details>` Blocks):** Niche theoretical derivations (Sherman-Morrison, Moore-Penrose pseudo-inverse algebra, Monkey Saddle, formal KKT proofs, Eve's Law, Uniform$(0, \theta)$ boundary MLE) are tucked neatly inside expandable `<details>` blocks—skip them on your first pass and expand them only when curious!

Day 1: Linear Algebra (Eigenvalues, Subspaces, SVD/PCA, Projections)

↑ Mục Lục

1.1 🌟 [Core — Bread & Butter] Eigenvalues, Eigenvectors & The 15-Second Shortcut Arsenal

💡 Intuition (Plain English First)

When you multiply a square matrix $A \in \mathbb{R}^{n \times n}$ by an arbitrary vector $x \in \mathbb{R}^n$, the output vector Ax usually gets **both rotated and stretched**—it points in a completely new direction.

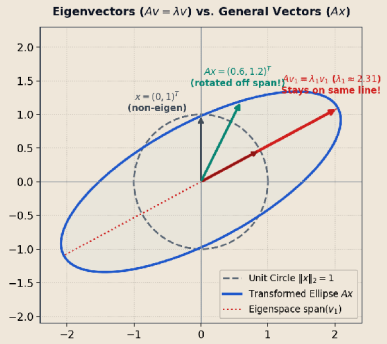
However, almost every square matrix has a few special "privileged axes" in space. If a vector v lies along one of these special axes, multiplying it by A **does not rotate it at all**—it simply stretches, shrinks, or flips v along that exact same line:

$$Av = \lambda v$$

- v **(Eigenvector):** The special direction/axis that refuses to rotate when multiplied by A.

- λ **(Eigenvalue):** The scalar stretch factor along that axis (e.g., $\lambda = 3$ stretches the vector $3\times$ longer; $\lambda = -1$ flips it backward; $\lambda = 0$ squashes that entire direction into the origin 0).

Why do we care in AI? Because if you apply a matrix A repeatedly ($A^k x$, like in a Recurrent Neural Network or multi-step optimization), any direction with $|\lambda| > 1$ explodes ($1.5^{100} \to \infty$), and any direction with $|\lambda| < 1$ vanishes ($0.5^{100} \to 0$). Furthermore, finding the eigenvalues of a 3×3 matrix on a timed MCQ by expanding $\det(A - \lambda I) = 0$ takes 4 minutes and is prone to sign errors—instead, exam designers expect you to use **Trace, Determinant, and Rank-1 shortcuts** to spot the eigenvalues in 15 seconds.



Eigenvectors (Av = λv) vs. General Vectors (Ax)

📐 The Math & Every Symbol Explained

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix (n rows, n columns) with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$:

1. Trace & Determinant Invariants (Always Check These First!)

$$\text{Tr}(A) = \sum_{i=1}^{n} A_{ii} = \sum_{i=1}^{n} \lambda_i, \qquad \det(A) = \prod_{i=1}^{n} \lambda_i$$

- $\text{Tr}(A)$ **(Trace):** The sum of the diagonal entries $A_{11} + A_{22} + \cdots + A_{nn}$. It **always equals the sum of all** n **eigenvalues**.
- $\det(A)$ **(Determinant):** Geometrically, how much A scales n-dimensional volume. It **always equals the product of all** n **eigenvalues**. (If even one eigenvalue is $\lambda_i = 0$, then $\det(A) = 0$, meaning A squashes space into a lower dimension and is **not invertible**).
- **For any** 2×2 **matrix**, the characteristic equation is simply:

$$\lambda^2 - \text{Tr}(A)\lambda + \det(A) = 0 \implies \lambda = \frac{\text{Tr}(A) \pm \sqrt{\text{Tr}(A)^2 - 4\det(A)}}{2}$$

2. Structural Shortcuts for 3×3 (and $n \times n$) Exam Matrices

- **Constant Row Sum Shortcut:** If every row of A adds up to the same number c, then multiplying A by the all-ones vector $\mathbf{1} = [1, 1, \ldots, 1]^T$ just adds each row up, giving $A\mathbf{1} = c\mathbf{1}$. Therefore, $\lambda = c$ **is guaranteed to be an eigenvalue** with right eigenvector $\mathbf{1}$! *(Even if every* ***column*** *adds up to* c*,* $\lambda = c$ *is still guaranteed to be an eigenvalue of* A *because* A *and* A^T *share the exact same eigenvalues).*

- **Rank-1 Matrix ($A = uv^T$):** Where $u, v \in \mathbb{R}^n$ are column vectors. Because every column of A is just a scalar multiple of u, A squashes all of n-dimensional space onto the single 1D line spanned by u.
  - Therefore, $n-1$ directions get squashed to zero ($n-1$ **eigenvalues equal** 0).
  - The single surviving direction is u, and its eigenvalue equals the trace: $\lambda_1 = \text{Tr}(uv^T) = u^T v$.
- **Shifted Rank-1 Matrix ($A = cI + uv^T$):** Adding cI simply adds $+c$ to every eigenvalue of uv^T. Thus the eigenvalues of $cI + uv^T$ are c (repeated $n-1$ times) and $c + u^T v$ (once).
- **Block-Triangular & Rank-1 Determinant Identities:**

$$\det \begin{bmatrix} A & B \\ 0 & D \end{bmatrix} = \det(A)\det(D), \qquad \det(I + uv^T) = 1 + u^T v$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Sherman-Morrison Rank-1 Inverse Formula**

If you already know A^{-1} and someone modifies A by adding a rank-1 matrix uv^T, you do **not** need to re-invert the matrix from scratch in $\mathcal{O}(n^3)$ time. The **Sherman-Morrison formula** updates the inverse in $\mathcal{O}(n^2)$ time:

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1} u}$$

(Used in Quasi-Newton methods like BFGS and Recursive Least Squares, though too algebra-heavy to compute by hand in a 90-second MCQ.)

3. How Operations Change Eigenvalues vs. Eigenvectors

Suppose $Av_i = \lambda_i v_i$. What happens when we transform A?

New Matrix	New Eigenvalues	New Eigenvectors	Why It Works Intuitively
$A + cI$ (Shift diagonal by c)	$\lambda_i + c$	**Unchanged (v_i)**	$(A + cI)v_i = Av_i + cv_i = (\lambda_i + c)v_i$
A^k (Multiply A k times)	λ_i^k	**Unchanged (v_i)**	Each multiplication by A scales v_i by another factor of λ_i. Also $\text{Tr}(A^k) = \sum \lambda_i^k$
A^{-1} (Inverse, if $\lambda_i \neq 0$)	$\frac{1}{\lambda_i}$	**Unchanged (v_i)**	If A stretches v_i by λ_i, undoing A shrinks v_i by $1/\lambda_i$
$P^{-1}AP$ (Similarity transform)	**Unchanged (λ_i)**	**Rotated to $P^{-1}v_i$**	Same linear transformation viewed in a new coordinate basis P

🔢 Step-by-Step Toy Example

Problem: Find the three eigenvalues of:

$$A = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Step 1 — Spot the structure (do NOT expand the 3×3 determinant!): Notice that every off-diagonal number is 1, and every diagonal number is $3 = 2 + 1$. We can split A into a diagonal matrix $2I$ plus an all-ones matrix

$\mathbf{1}\mathbf{1}^T$:

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = 2I + \mathbf{1}\mathbf{1}^T$$

Step 2 — Find the eigenvalues of the rank-1 all-ones matrix $\mathbf{1}\mathbf{1}^T$:

- Since all 3 rows of $\mathbf{1}\mathbf{1}^T$ are identical, it has rank 1.
- A 3×3 rank-1 matrix always has two eigenvalues equal to 0, and one eigenvalue equal to its trace:

$$\text{Tr}(\mathbf{1}\mathbf{1}^T) = 1 + 1 + 1 = 3$$

- So the eigenvalues of $\mathbf{1}\mathbf{1}^T$ are $\{3, 0, 0\}$.

Step 3 — Shift by $+2I$:

- Adding $2I$ adds $+2$ to every eigenvalue:

$$\{3 + 2,\ 0 + 2,\ 0 + 2\} = \{\mathbf{5}, \mathbf{2}, \mathbf{2}\}$$

Step 4 — 3-Second Sanity Check:

- Sum of our eigenvalues: $5 + 2 + 2 = 9$.
- Trace of A: $3 + 3 + 3 = 9$. Matches! Total time: **10 seconds.**

🎯 Exam Takeaway & Fast Shortcuts

- Whenever an MCQ asks for eigenvalues of a 3×3 matrix, **eliminate wrong options first** by checking if the options sum to $\text{Tr}(A)$! Often, only 1 of the 4 multiple-choice options will even add up to the trace.
- Watch out for the **Similarity Transform Trap:** $B = P^{-1}AP$ has the **exact same eigenvalues, trace, determinant, and rank** as A, but it does **not** have the same eigenvectors ($v_i \mapsto P^{-1}v_i$).

1.2 🌟 [Core — Bread & Butter] Rank & The Four Fundamental Subspaces (Image Space $\text{Im}(A)$ & Kernel $\text{Ker}(A)$)

💡 Intuition (Plain English First)

Let's standardize our matrix dimensions: throughout this guide, a rectangular matrix $A \in \mathbb{R}^{n \times m}$ always has n **rows** and m **columns**:

$$A = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \Bigg\} n \text{ rows} \quad \text{with } m \text{ columns}$$

When A multiplies an input vector x to produce an output vector $y = Ax$:

- **Input x must have m entries ($x \in \mathbb{R}^m$):** Because each of the m entries of x weights one of the m columns of A.

- **Output $y = Ax$ has n entries ($y \in \mathbb{R}^n$):** Because each of the n rows of A produces one dot-product entry in y.

So A is a machine that maps **Domain $\mathbb{R}^m$ (m columns) $\longrightarrow$ Codomain $\mathbb{R}^n$ (n rows)**.

What is the **rank** $r = \text{rank}(A)$? It is the number of **truly independent directions** (columns or rows) that survive when A transforms space. Because you can't have more independent directions than the number of rows or columns, $r \leq \min(n, m)$.

This splits the input space $\mathbb{R}^m$ and output space $\mathbb{R}^n$ into **Four Fundamental Subspaces**:

1. **Image Space (Column Space) $\text{Im}(A) = \mathcal{C}(A) \subseteq \mathbb{R}^n$:** The set of all possible outputs $y = Ax$ that the matrix can actually reach. Since every output Ax is a linear combination of the columns of A, the Image Space is spanned by the columns of A and has dimension r.
2. **Kernel (Nullspace) $\text{Ker}(A) = \mathcal{N}(A) \subseteq \mathbb{R}^m$:** The set of all input vectors $x \in \mathbb{R}^m$ that get **completely crushed to zero** ($Ax = 0$). Since you start with m input dimensions and r of them survive into the Image Space, the remaining $m - r$ input dimensions must get crushed to zero!
3. **Row Space $\text{Im}(A^T) = \mathcal{C}(A^T) \subseteq \mathbb{R}^m$:** The r-dimensional subspace of the input domain $\mathbb{R}^m$ that *survives* multiplication by A without being crushed to zero. It is strictly perpendicular (orthogonal) to the Kernel $\text{Ker}(A)$.
4. **Left Nullspace (Cokernel) $\text{Ker}(A^T) = \mathcal{N}(A^T) \subseteq \mathbb{R}^n$:** The $(n - r)$-dimensional subspace of the output space $\mathbb{R}^n$ that is **unreachable** by Ax and sits strictly perpendicular to the Image Space $\text{Im}(A)$.

📐 The Math & Every Symbol Explained

For any matrix $A \in \mathbb{R}^{n \times m}$ (n rows, m columns) with $\text{rank}(A) = r$:

$$\underbrace{\dim(\text{Im}(A))}_{\text{Rank } r} + \underbrace{\dim(\text{Ker}(A))}_{\text{Nullity } m-r} = \mathbf{m} \text{ (Number of Columns!)} \qquad \text{(Rank-Nullity Theorem)}$$

Subspace Name	School / Math Notation	Lives In	Dimension	Geometric Meaning
Image Space (Column Space)	$\text{Im}(A) = \mathcal{C}(A) = \{Ax : x \in \mathbb{R}^m\}$	Output $\mathbb{R}^n$	r	All reachable outputs $y = Ax$
Kernel (Nullspace)	$\text{Ker}(A) = \mathcal{N}(A) = \{x \in \mathbb{R}^m : Ax = 0\}$	Input $\mathbb{R}^m$	$m - r$	All inputs crushed to 0; $\text{Ker}(A) \perp \text{Im}(A^T)$
Row Space	$\text{Im}(A^T) = \mathcal{C}(A^T)$	Input $\mathbb{R}^m$	r	Input directions mapped 1-to-1 onto $\text{Im}(A)$
Left Nullspace (Cokernel)	$\text{Ker}(A^T) = \mathcal{N}(A^T) = \{y \in \mathbb{R}^n : A^T y = 0\}$	Output $\mathbb{R}^n$	$n - r$	Unreachable output directions; $\text{Ker}(A^T) \perp \text{Im}(A)$

High-Frequency Rank Identities

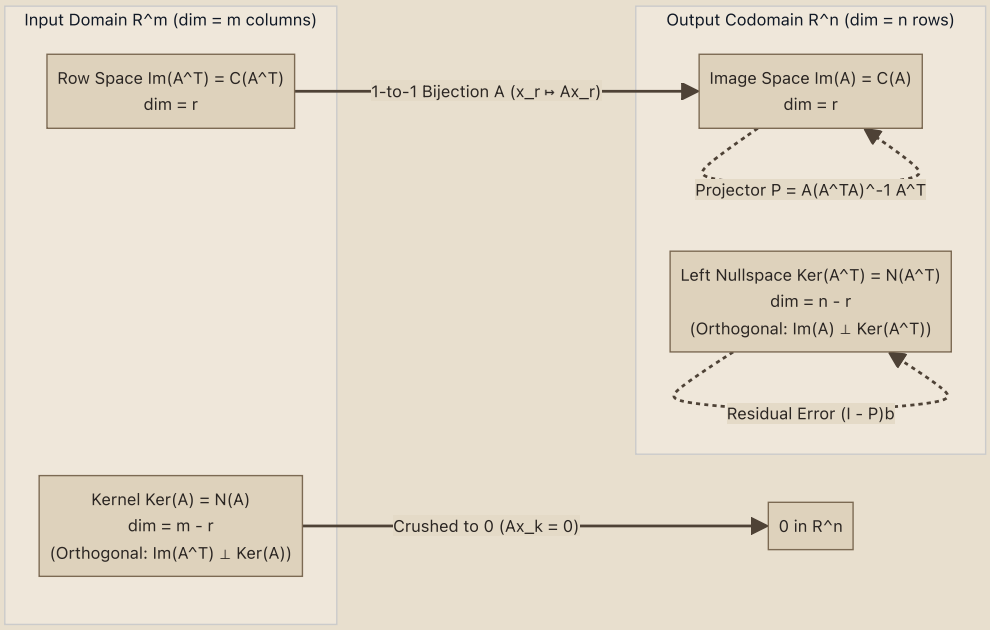
- **Gram Matrix Rank Invariance:** For *any* real matrix $A \in \mathbb{R}^{n \times m}$:

$$\text{rank}(A) = \text{rank}(A^T) = \text{rank}(A^T A) = \text{rank}(A A^T) = r$$

(Why? Because $A^TAx = 0 \implies x^TA^TAx = \|Ax\|_2^2 = 0 \implies Ax = 0$, so A and A^TA have the exact same Kernel $\text{Ker}(A^TA) = \text{Ker}(A)$!)

- **Rank of a Sum:** $|\text{rank}(A) - \text{rank}(B)| \leq \text{rank}(A+B) \leq \text{rank}(A) + \text{rank}(B)$.
- **Sylvester's Rank Inequality (Rank of a Product):** For $A \in \mathbb{R}^{n \times k}$ and $B \in \mathbb{R}^{k \times m}$:

$$\text{rank}(A) + \text{rank}(B) - k \leq \text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$$



🔢 Step-by-Step Toy Example

Problem: Suppose A is a 3×4 matrix ($n = 3$ rows, $m = 4$ columns):

$$A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Find $\text{rank}(A)$ and the dimension of all four subspaces.

Step 1 — Find the Rank r:

- Look at the rows: Row 1 and Row 2 are independent (pivots in Column 1 and Column 2), while Row 3 is all zeros.
- Therefore, $\text{rank}(A) = r = 2$.

Step 2 — Find the Image Space $\text{Im}(A)$ and Kernel $\text{Ker}(A)$:

- **Image Space $\text{Im}(A)$:** Lives in $\mathbb{R}^3$ ($n = 3$ rows). Its dimension is $r = \mathbf{2}$ (specifically, the xy-plane where the 3rd coordinate is 0).
- **Kernel $\text{Ker}(A)$:** Lives in $\mathbb{R}^4$ ($m = 4$ columns). By Rank-Nullity, $\dim(\text{Ker}(A)) = m - r = 4 - 2 = \mathbf{2}$. (There are 2 free variables, x_3 and x_4, that you can vary while keeping $Ax = 0$).

Step 3 — Find the Row Space $\text{Im}(A^T)$ and Left Nullspace $\text{Ker}(A^T)$:

- **Row Space** $\text{Im}(A^T)$**:** Lives in $\mathbb{R}^4$ ($m = 4$ columns), with dimension $r = \mathbf{2}$.
- **Left Nullspace** $\text{Ker}(A^T)$**:** Lives in $\mathbb{R}^3$ ($n = 3$ rows), with dimension $n - r = 3 - 2 = \mathbf{1}$ (specifically, the z-axis $[0, 0, c]^T$, which is perpendicular to $\text{Im}(A)$).

🎯 Exam Takeaway & Fast Shortcuts

- **The #1 Rank-Nullity Trap:** When given $A \in \mathbb{R}^{n \times m}$ (n rows, m columns), remember that $\dim(\text{Im}(A)) + \dim(\text{Ker}(A)) = \mathbf{m}$ (**number of columns / inputs**), NEVER n!
- If a linear system $Ax = b$ has at least one particular solution x_p, then the set of **all** solutions is the affine subspace $x = x_p + x_k$ for any $x_k \in \text{Ker}(A)$. Thus, a solution is **unique if and only if** $\text{Ker}(A) = \{0\}$ (i.e., $r = m$, full column rank).

1.3 🌟 [Core — Bread & Butter] Symmetric, Orthogonal, and Positive Semi-Definite (PSD) Matrices

💡 Intuition (Plain English First)

General square matrices can behave badly: they can have complex eigenvalues, non-perpendicular eigenvectors, or even be missing eigenvectors altogether ("defective" matrices like $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$).

Fortunately, in Machine Learning, three families of matrices behave *beautifully*:

1. **Symmetric Matrices ($A = A^T$):** Examples include Covariance matrices, Hessian matrices ($\nabla^2 f$), and Kernel/Gram matrices ($X^T X$). A symmetric matrix **never rotates vectors into complex space**: its eigenvalues are guaranteed to be **100% real numbers**, and its eigenvectors are guaranteed to form a **perpendicular (orthogonal) coordinate system**!
2. **Orthogonal Matrices ($Q^T Q = QQ^T = I$):** These are pure **rigid rotations or mirror reflections**. Multiplying a vector x by Q rotates x without changing its length ($\|Qx\|_2 = \|x\|_2$) and without changing the angle between any two vectors.
3. **Positive Definite ($A \succ 0$) & Positive Semi-Definite ($A \succeq 0$) Matrices:** These are the matrix equivalent of "strictly positive numbers ($a > 0$)" and "non-negative numbers ($a \geq 0$)." Geometrically, if $A \succ 0$, the quadratic surface $f(x) = x^T Ax$ is an **upward-facing bowl** with a single global minimum at $x = 0$.

📐 The Math & Every Symbol Explained

1. Real Symmetric Matrix ($A = A^T \in \mathbb{R}^{n \times n}$)

- **Spectral Theorem:** Every real symmetric matrix can be factored as:

$$A = Q\Lambda Q^T = \sum_{i=1}^{n} \lambda_i q_i q_i^T$$

  - $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$**:** Diagonal matrix of real eigenvalues ($\lambda_i \in \mathbb{R}$).

- $Q = [q_1, \ldots, q_n] \in \mathbb{R}^{n \times n}$: Orthogonal matrix whose columns are orthonormal eigenvectors ($q_i^T q_j = \delta_{ij}$, so $Q^{-1} = Q^T$).

- **Rayleigh Quotient Bounds:** If you plug any unit vector x ($\|x\|_2 = 1$) into $x^T A x$, the output is bounded between the smallest and largest eigenvalue:

  $$\lambda_{\min}(A) \leq \frac{x^T A x}{x^T x} \leq \lambda_{\max}(A)$$

  (The maximum $\lambda_{\max}$ is achieved when $x = q_{\max}$, and the minimum $\lambda_{\min}$ when $x = q_{\min}$.)

2. Orthogonal Matrix ($Q^T Q = Q Q^T = I_n$)

- **Length & Angle Preservation:** $\|Qx\|_2^2 = (Qx)^T(Qx) = x^T Q^T Q x = x^T I x = \|x\|_2^2$.
- **Determinant & Eigenvalues:** $\det(Q) = +1$ (pure rotation) or -1 (rotation + reflection). Every eigenvalue has magnitude $|\lambda_i| = 1$ (can be complex $e^{\pm i\theta}$ for a 2D rotation by angle θ).

3. Positive Definite ($A \succ 0$) vs. Positive Semi-Definite ($A \succeq 0$)

For a **symmetric** matrix $A = A^T \in \mathbb{R}^{n \times n}$:

Property	Positive Definite ($A \succ 0$, PD)	Positive Semi-Definite ($A \succeq 0$, PSD)
Quadratic Form	$x^T A x > 0$ for **all** $x \neq 0$	$x^T A x \geq 0$ for **all** x
Eigenvalues	All $\lambda_i > 0$ (strictly positive)	All $\lambda_i \geq 0$ (non-negative)
Invertibility	Always invertible ($\det(A) > 0$)	May be singular if any $\lambda_i = 0$
Leading Principal Minors	All $k \times k$ top-left sub-determinants > 0 (**Sylvester's Criterion**)	All principal minors ≥ 0 *(see warning below)*
Factorization	Unique Cholesky $A = LL^T$ ($L_{ii} > 0$)	$A = X^T X$ for some $X \in \mathbb{R}^{n \times m}$

> ⚠ **WARNING**
>
> **Two Famous PSD/PD Exam Traps:**
>
> 1. **Symmetry is Required for Eigenvalue/Minor Tests:** A non-symmetric matrix like $M = \begin{bmatrix} 1 & 4 \\ 0 & 1 \end{bmatrix}$ has positive eigenvalues $\{1, 1\}$ and positive leading minors, yet for $x = [1, -1]^T$, $x^T M x = 1 - 4 + 1 = -2 < 0$! Always check $x^T M x = x^T \left(\frac{M + M^T}{2} \right) x$.
> 2. **Gram Matrix Guarantee:** For *any* rectangular matrix $X \in \mathbb{R}^{n \times m}$, the matrix $G = X^T X \in \mathbb{R}^{m \times m}$ is **always symmetric PSD** because $x^T (X^T X) x = (Xx)^T (Xx) = \|Xx\|_2^2 \geq 0$. Adding ridge penalty λI_m ($\lambda > 0$) shifts every eigenvalue from ≥ 0 to $\geq \lambda > 0$, making $X^T X + \lambda I_m$ **strictly Positive Definite and guaranteed invertible**!

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Why Sylvester's Leading Minors Fail for PSD**

For strict Positive Definiteness ($A \succ 0$), checking only the n **leading** (top-left) principal minors > 0 is necessary and sufficient. However, for **Positive Semi-Definiteness ($A \succeq 0$)**, checking only top-left minors ≥ 0 is **not enough**! For example, $A = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$ has top-left 1×1 minor $= 0$ and 2×2 determinant $= 0$, yet its eigenvalues are $\{0, -1\}$ (not PSD!). To test PSD via minors, all $2^n - 1$ principal minors must be ≥ 0. On timed exams, simply check $\lambda_i \geq 0$ or $x^T Ax \geq 0$.

🔢 Step-by-Step Toy Example

Problem: For what values of c is the symmetric matrix $A = \begin{bmatrix} 4 & c \\ c & 9 \end{bmatrix}$ strictly Positive Definite ($A \succ 0$)?

Step 1 — Check Symmetry:

- $A = A^T$ is symmetric.

Step 2 — Apply Sylvester's Criterion (Top-Left Leading Principal Minors > 0):

- **1st leading minor (1×1 top-left entry):** $A_{11} = 4 > 0$ ✔
- **2nd leading minor (2×2 determinant):**

$$\det(A) = (4)(9) - c^2 = 36 - c^2 > 0$$

Step 3 — Solve for c:

- $c^2 < 36 \implies \mathbf{-6 < c < 6}$.
- *(If $c = \pm 6$, then $\det(A) = 0$ and $\text{Tr}(A) = 13$, so $\lambda = \{13, 0\}$, making A Positive Semi-Definite $A \succeq 0$ instead of strictly PD.)*

🎯 Exam Takeaway & Fast Shortcuts

- To check if a 2×2 symmetric matrix A is strictly Positive Definite ($A \succ 0$), you only need two instant checks: $A_{11} > 0$ **AND** $\det(A) > 0$ (which is equivalent to $\text{Tr}(A) > 0$ and $\det(A) > 0$).

1.4 🌟 [Core — Bread & Butter] Singular Value Decomposition (SVD: $A = U\Sigma V^T$), Pseudo-Inverse & PCA

💡 Intuition (Plain English First)

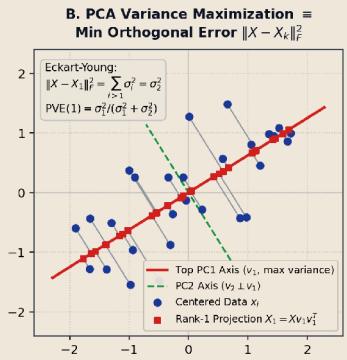
Let's connect **PCA** and **SVD** from the ground up, because once you see how they fit together, 5 different linear algebra formulas become one single mental picture.

Part A: What is PCA doing? (The Camera Angle Analogy)

Imagine you have a dataset of $N = 1,000$ houses with $d = 2$ features: $x_1 =$ Square Footage and $x_2 =$ Number of Rooms. If you plot these $1,000$ houses on a 2D scatter plot, they don't form a round circle—they form a **tilted**

cigar-shaped cloud because bigger houses have more rooms.

- **The Problem:** The two features are redundant. If you need to compress those 2 features down to **1 single number** per house while losing as little information as possible, what should you do?
- **PCA's Solution:** Instead of using the original horizontal/vertical axes (x_1, x_2), **rotate your coordinate axes** so that:
  1. **Principal Component 1 (v_1):** Points directly along the **longest axis of the cigar** (the direction of **maximum variance / spread**).
  2. **Principal Component 2 (v_2):** Points perpendicular to v_1 across the narrow width of the cigar (mostly noise). By projecting every data point onto v_1, you keep the maximum possible variance (which is mathematically identical to minimizing the squared perpendicular reconstruction error!).



PCA Variance Maximization & Proportion of Variance Explained

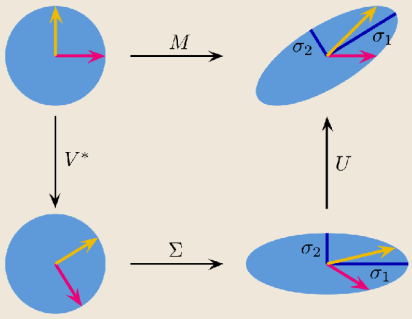
Part B: What is SVD doing? (Rotate → Stretch → Rotate)

Eigendecomposition ($A = Q\Lambda Q^T$) only works on **square symmetric** matrices. What if your matrix $A \in \mathbb{R}^{n \times m}$ (n rows, m columns) is **rectangular**? **Singular Value Decomposition (SVD)** says that **every rectangular matrix** $A \in \mathbb{R}^{n \times m}$ can be broken down into three simple geometric steps:

$$A = U\Sigma V^T$$

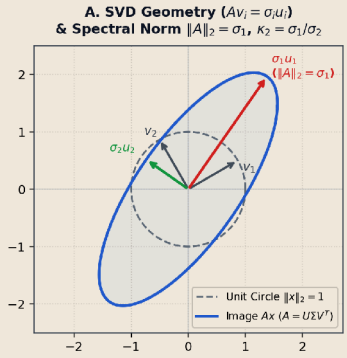
When you multiply a vector $x \in \mathbb{R}^m$ by A ($y = U\Sigma V^T x$):

1. $V^T x$ **(Rotate in Input Space $\mathbb{R}^m$):** V^T is an $m \times m$ orthogonal rotation matrix that aligns x with the input principal axes $v_1, \ldots, v_m$.
2. $\Sigma(V^T x)$ **(Stretch along the axes):** Σ is an $n \times m$ diagonal matrix of non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ that stretches the i-th axis by σ_i (and pads/drops zeros to change dimension from m to n).
3. $U(\Sigma V^T x)$ **(Rotate in Output Space $\mathbb{R}^n$):** U is an $n \times n$ orthogonal rotation matrix that places the stretched vector onto its final orientation $u_1, \ldots, u_n$ in the output space $\mathbb{R}^n$.



$$M = U \cdot \Sigma \cdot V^*$$

Step-by-Step SVD Transformation: Rotate by V → Stretch by Σ → Rotate by U*



SVD Geometric Decomposition: Unit Circle to Stretched Ellipse

Part C: How SVD and PCA Are the Exact Same Thing!

Suppose $X \in \mathbb{R}^{N \times d}$ is your mean-centered data matrix (N rows = data points, d columns = features). In PCA, we form the sample covariance matrix $S = \frac{1}{N-1} X^T X \in \mathbb{R}^{d \times d}$ and find its eigenvectors. Now plug the SVD of $X = U\Sigma V^T$ into $X^T X$:

$$X^T X = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T \underbrace{U^T U}_{I} \Sigma V^T = V\Sigma^2 V^T$$

Look at $X^T X = V\Sigma^2 V^T$:

1. The **right singular vectors** V of X are **literally the PCA principal component directions (eigenvectors of $X^T X$)**!
2. The **eigenvalues of** $X^T X$ are $\lambda_i = \sigma_i^2$ (the squared singular values of X), so the covariance eigenvalues are $\lambda_i(S) = \frac{\sigma_i^2}{N-1}$.

📐 The Math & Every Symbol Explained

1. Full vs. Dyadic SVD ($A \in \mathbb{R}^{n \times m}$, n rows, m columns, rank r)

$$A = U\Sigma V^T = \sum_{i=1}^{r} \sigma_i u_i v_i^T$$

- $U \in \mathbb{R}^{n \times n}$ **(Left Singular Vectors):** Orthogonal matrix ($U^T U = I_n$) whose columns $u_i \in \mathbb{R}^n$ are eigenvectors of $AA^T \in \mathbb{R}^{n \times n}$. First r columns span $\text{Im}(A)$; last $n - r$ columns span $\text{Ker}(A^T)$.
- $\Sigma \in \mathbb{R}^{n \times m}$ **(Singular Values):** Rectangular diagonal matrix with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ on the diagonal, where $\sigma_i = \sqrt{\lambda_i(A^T A)} = \sqrt{\lambda_i(AA^T)}$.
- $V \in \mathbb{R}^{m \times m}$ **(Right Singular Vectors):** Orthogonal matrix ($V^T V = I_m$) whose columns $v_i \in \mathbb{R}^m$ are eigenvectors of $A^T A \in \mathbb{R}^{m \times m}$. First r columns span $\text{Im}(A^T)$; last $m - r$ columns span $\text{Ker}(A)$.

▼ 🔬 **Optional Deep-Dive: Moore-Penrose Pseudo-Inverse (A^+) for Rectangular / Rank-Deficient Matrices (Click to Expand)**

2. Moore-Penrose Pseudo-Inverse ($A^+ \in \mathbb{R}^{m \times n}$)

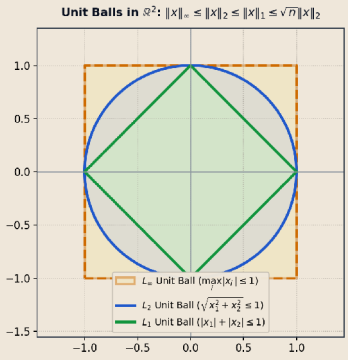
How do you "invert" a rectangular or rank-deficient matrix A? You invert the 3 SVD steps in reverse order—rotating back by V, inverting only the non-zero singular values $\sigma_i \mapsto \frac{1}{\sigma_i}$ (leaving $0 \mapsto 0$), and rotating by U^T:

$$A^+ = V\Sigma^+ U^T = \sum_{i=1}^{r} \frac{1}{\sigma_i} v_i u_i^T \in \mathbb{R}^{m \times n}$$

- If A has **full column rank** ($r = m$): $A^+ = (A^T A)^{-1} A^T$ (satisfies $A^+ A = I_m$).
- If A has **full row rank** ($r = n$): $A^+ = A^T (AA^T)^{-1}$ (satisfies $AA^+ = I_n$).
- For **any** linear system $Ax \approx b$ (even when underdetermined with infinitely many solutions!), $x^* = A^+ b$ gives the unique solution that minimizes $\|Ax - b\|_2^2$ and has the **smallest possible** L_2 **norm** $\|x^*\|_2$.

3. Vector & Matrix Norms, Condition Number & Eckart-Young-Mirsky Low-Rank Approximation

- **Vector** L_p **Norms** ($x \in \mathbb{R}^n$): $\|x\|_1 = \sum_i |x_i|$ (Manhattan), $\|x\|_2 = \sqrt{\sum_i x_i^2}$ (Euclidean), $\|x\|_\infty = \max_i |x_i|$ (Max norm), obeying $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1 \leq \sqrt{n}\|x\|_2$.



Vector Unit Balls in R^2: L1 Diamond, L2 Circle, L-infinity Square

- **Spectral Norm** (L_2 **Operator Norm**): Maximum stretch factor of A on any unit vector:

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sigma_1 \quad \text{(Largest singular value!)}$$

- **Frobenius Norm:** Square root of the sum of all squared matrix entries:

$$\|A\|_F = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{m} A_{ij}^2} = \sqrt{\text{Tr}(A^T A)} = \sqrt{\sum_{i=1}^{r} \sigma_i^2}$$

- **Condition Number:** $\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$ (measures how elongated the matrix is; if $\kappa_2(A) \gg 1$, solving $Ax = b$ is numerically unstable and Gradient Descent converges slowly).
- **Eckart-Young-Mirsky Theorem (Best Rank-k Compression):** If you approximate A by keeping only the top $k < r$ singular values ($A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$), the exact approximation error is:

$$\|A - A_k\|_2 = \sigma_{k+1}, \qquad \|A - A_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}$$

- **PCA Proportion of Variance Explained (PVE) by Top k Components:**

$$\text{PVE}(k) = \frac{\sum_{i=1}^{k} \lambda_i(S)}{\sum_{j=1}^{d} \lambda_j(S)} = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{j=1}^{d} \sigma_j^2}$$

🔢 Step-by-Step Toy Example

Problem: A centered data matrix $X \in \mathbb{R}^{100 \times 4}$ ($N = 100$ samples, $d = 4$ features) has singular values $\sigma_1 = 6,\ \sigma_2 = 3,\ \sigma_3 = 2,\ \sigma_4 = 1$.

1. What are the spectral norm $\|X\|_2$ and Frobenius norm $\|X\|_F$?
2. If we compress X to a rank-2 matrix X_2 using SVD, what are the errors $\|X - X_2\|_2$ and $\|X - X_2\|_F$?
3. What percentage of total variance is explained by the first $k = 2$ Principal Components?

Step 1 — Norms of X:

- Spectral norm $\|X\|_2 = \sigma_1 = \mathbf{6}$.
- Frobenius norm $\|X\|_F = \sqrt{\sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \sigma_4^2} = \sqrt{36 + 9 + 4 + 1} = \sqrt{\mathbf{50}}$.

Step 2 — Rank-2 Truncation Errors ($k = 2$):

- X_2 keeps $\sigma_1 = 6$ and $\sigma_2 = 3$, and drops $\sigma_3 = 2$ and $\sigma_4 = 1$.
- Spectral error $\|X - X_2\|_2 = \sigma_{k+1} = \sigma_3 = \mathbf{2}$.
- Frobenius error $\|X - X_2\|_F = \sqrt{\sigma_3^2 + \sigma_4^2} = \sqrt{2^2 + 1^2} = \sqrt{\mathbf{5}}$.

Step 3 — PCA Proportion of Variance Explained (PVE for $k = 2$):

- **Crucial Check:** Variance is proportional to σ_i^2 (squared singular values!), NOT σ_i:

$$\text{PVE}(2) = \frac{\sigma_1^2 + \sigma_2^2}{\sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \sigma_4^2} = \frac{36 + 9}{36 + 9 + 4 + 1} = \frac{45}{50} = \mathbf{90\%}$$

🎯 Exam Takeaway & Fast Shortcuts

- **The #1 PCA/SVD Trap:** Singular values of X are σ_i, whereas eigenvalues of $X^T X$ (and variance along each principal component) are proportional to σ_i^2! When an exam gives you singular values $\sigma = \{6, 3, 2, 1\}$ and asks for Variance Explained, **always square them first** $(36, 9, 4, 1)$ before dividing!

1.5 🌟 [Core — Bread & Butter] Orthogonal Projections & Least Squares Geometry

💡 Intuition (Plain English First)

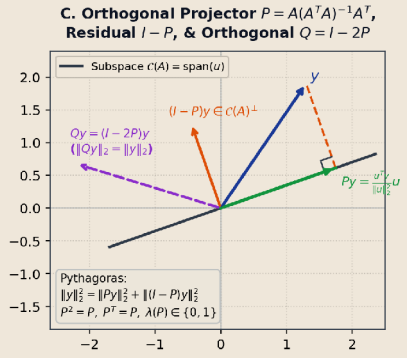
Suppose you have an overdetermined linear system $Ax = b$ where $A \in \mathbb{R}^{n \times m}$ is a tall matrix ($n > m$, e.g., 100 equations/rows and only 2 unknowns/columns).

- All possible outputs Ax live inside the m-dimensional **Image Space** $\text{Im}(A)$ (a flat 2D plane sitting inside 100-dimensional space $\mathbb{R}^{100}$).
- Your target vector $b \in \mathbb{R}^n$ almost certainly does **not** lie on that flat plane—so $Ax = b$ has **no exact solution**.

How do we find the best approximate solution $\hat{x}$ that minimizes the squared error $\|Ax - b\|_2^2$? Think of shining a flashlight straight down (perpendicularly) onto the plane $\text{Im}(A)$:

1. The **closest point** on the plane $\text{Im}(A)$ to b is the **orthogonal shadow (projection)** $\hat{b} = Pb = A\hat{x}$.
2. Because $\hat{b}$ is directly underneath b, the **residual error vector** $r = b - A\hat{x}$ **must stick straight up, 90° perpendicular to the plane** $\text{Im}(A)$!
3. Saying "$b - A\hat{x}$ is perpendicular to every column of A" in math is simply:

$$A^T(b - A\hat{x}) = 0 \implies A^T A\hat{x} = A^T b \quad \text{(The Normal Equations!)}$$



Orthogonal Projection hat(b) = Pb & Residual r = (I - P)b
Perpendicular to Im(A)

📐 The Math & Every Symbol Explained

1. Projection onto a Single 1D Vector $u \in \mathbb{R}^n$

$$P_u = \frac{uu^T}{u^Tu} \in \mathbb{R}^{n \times n}, \qquad \text{proj}_u(y) = P_u y = \left(\frac{u^Ty}{\|u\|_2^2}\right)u$$

- uu^T **($n \times n$ outer product matrix):** A rank-1 matrix.
- $u^Tu = \|u\|_2^2$ **(scalar inner product):** Normalizes for the length of u.

2. Projection onto Image Space $\text{Im}(A)$ (for full-column-rank $A \in \mathbb{R}^{n \times m}$, $r = m$)

Solving $A^TA\hat{x} = A^Tb$ gives $\hat{x} = (A^TA)^{-1}A^Tb$. Multiplying by A gives the projected vector $\hat{b} = A\hat{x} = Pb$, where the **Orthogonal Projection Matrix** $P \in \mathbb{R}^{n \times n}$ is:

$$P = A(A^TA)^{-1}A^T \in \mathbb{R}^{n\times n}$$

3. The 5 Algebraic Laws of Any Orthogonal Projection Matrix P

If a question asks "*Which of the following is true about* $P = A(A^TA)^{-1}A^T$*?*", test these 5 geometric properties:

1. **Symmetric ($P^T = P$):** Because it is an *orthogonal* (perpendicular) projection.
2. **Idempotent ($P^2 = P$):** Projecting a shadow that is *already on the floor* onto the floor a second time doesn't move it ($P(Py) = Py$).
3. **Eigenvalues are strictly 0 or 1:** Since $P^2v = Pv \implies \lambda^2 = \lambda \implies \lambda \in \{0, 1\}$.
   - Any vector already inside $\text{Im}(A)$ stays unchanged ($\lambda = 1$, multiplicity r).
   - Any vector perpendicular to $\text{Im}(A)$ (in $\text{Ker}(A^T)$) gets squashed to zero ($\lambda = 0$, multiplicity $n - r$).
4. **Trace equals Rank ($\text{Tr}(P) = \text{rank}(P) = r$):** Because the trace is the sum of eigenvalues (r ones and $n - r$ zeros)!
5. **Complementary Residual Projector ($I - P$):** $I - P$ is ALSO an orthogonal projection matrix! It projects onto the Left Nullspace $\text{Ker}(A^T) = \text{Im}(A)^\perp$, and $P(I - P) = P - P^2 = 0$.

🔢 Step-by-Step Toy Example

Problem: Let $u = \begin{bmatrix}1\\2\\2\end{bmatrix} \in \mathbb{R}^3$. Construct the 3×3 projection matrix P_u onto the line spanned by u, verify its trace and eigenvalues, and find the projection of $y = \begin{bmatrix}3\\3\\0\end{bmatrix}$ onto u.

Step 1 — Compute u^Tu and P_u:

- $u^Tu = 1^2 + 2^2 + 2^2 = 9$.
- Outer product uu^T:

$$P_u = \frac{1}{9}\begin{bmatrix}1\\2\\2\end{bmatrix}\begin{bmatrix}1 & 2 & 2\end{bmatrix} = \frac{1}{9}\begin{bmatrix}1 & 2 & 2\\2 & 4 & 4\\2 & 4 & 4\end{bmatrix}$$

Step 2 — Verify Trace & Eigenvalues:

- $\text{Tr}(P_u) = \frac{1+4+4}{9} = \frac{9}{9} = \mathbf{1}$ (matches the dimension $r = 1$ of the line!).
- Eigenvalues of P_u are $\{1, 0, 0\}$.

Step 3 — Project $y = [3, 3, 0]^T$ onto u:

- Instead of multiplying the full 3×3 matrix, use the dot-product scalar $\frac{u^Ty}{u^Tu}$:

$$\frac{u^Ty}{u^Tu} = \frac{(1)(3) + (2)(3) + (2)(0)}{9} = \frac{9}{9} = 1$$

- Thus $\text{proj}_u(y) = 1 \cdot u = \begin{bmatrix}1\\2\\2\end{bmatrix}$.

- Check that the residual $r = y - \text{proj}_u(y) = [2, 1, -2]^T$ is perpendicular to u: $u^T r = (1)(2) + (2)(1) + (2)(-2) = 0$ ✔.

🎯 Exam Takeaway & Fast Shortcuts

- If an MCQ gives you a 100×4 full-rank matrix A and asks for the **trace** or **sum of eigenvalues** of $P = A(A^TA)^{-1}A^T$, do zero matrix multiplication! Use the cyclic trace property:

$$\text{Tr}\big(A(A^TA)^{-1}A^T\big) = \text{Tr}\big(\underbrace{A^TA(A^TA)^{-1}}_{I_4}\big) = \text{Tr}(I_4) = \mathbf{4}$$

Day 2: Multivariable Matrix Calculus & Optimization

↑ Mục Lục

2.1 🌟 [Core — Bread & Butter] Matrix Calculus Without Memorization

💡 Intuition (Plain English First)

In single-variable calculus, you know three simple rules by heart:

1. The derivative of a linear function $f(x) = ax$ is a (and second derivative is 0).
2. The derivative of a quadratic function $f(x) = ax^2$ is $2ax$ (and second derivative is $2a$).
3. The derivative of $f(x) = (ax - b)^2$ is $2a(ax - b)$.

Matrix calculus is the exact same thing, with one single rule to remember: **vectors and matrices don't commute ($x^TA \neq Ax$), so you transpose matrices to make the dimensions match the variable you are differentiating with respect to!** In Machine Learning, we always use the **Gradient / Denominator Layout**: if $x \in \mathbb{R}^n$ is a column vector of shape $(n \times 1)$ and $f(x) \in \mathbb{R}$ is a scalar loss, then the gradient $\nabla_x f(x)$ must **also be a column vector of the exact same shape** $(n \times 1)$ so that we can subtract it in Gradient Descent ($x \leftarrow x - \eta\nabla_x f(x)$).

📐 The Math & Every Symbol Explained

Let $x, a \in \mathbb{R}^n$ be column vectors $(n \times 1)$, $b \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, and $W, M \in \mathbb{R}^{n \times m}$ (n rows, m columns):

Scalar Function f	Single-Var Analogy	Matrix Gradient ∇f	Hessian $\nabla^2 f$ & Exam Notes
$f(x) = a^Tx = x^Ta$	$\frac{d}{dx}(ax) = a$	$\nabla_x f = a$	$\nabla_x^2 f = 0$. Linear dot product.
$f(x) = x^TAx$	$\frac{d}{dx}(ax^2) = 2ax$	$\nabla_x f = (A + A^T)x$	$\nabla_x^2 f = A + A^T$. **Equals $2Ax$ ONLY if $A = A^T$ is symmetric!**
$f(x) = \|Ax - b\|_2^2$	$\frac{d}{dx}(ax - b)^2 = 2a(ax - b)$	$\nabla_x f = 2A^T(Ax - b)$	$\nabla_x^2 f = 2A^TA$. Setting $\nabla_x f = 0 \implies A^TAx = A^Tb$.

Scalar Function f	Single-Var Analogy	Matrix Gradient ∇f	Hessian $\nabla^2 f$ & Exam Notes
$f(W) = \text{Tr}(M^T W)$	$\sum_{i,j} M_{ij} W_{ij}$	$\nabla_W f = M$	$\text{Tr}(M^T W) = \langle M, W \rangle_F$ is just the elementwise dot product of two $n \times m$ matrices!
$f(A) = \log \lvert\det(A)\rvert$	$\frac{d}{dx} \ln x = x^{-1}$	$\nabla_A f = (A^{-1})^T$	For symmetric PD covariance Σ, $\nabla_\Sigma \log\det(\Sigma) = \Sigma^{-1}$ (used in Gaussian MLE).

🔢 Step-by-Step Toy Example

Problem: Let $A = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$ (notice $A \neq A^T$ is NOT symmetric!) and $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. Compute $\nabla_x (x^T A x)$ and verify why the naive formula "$2Ax$" is wrong.

Step 1 — Expand $f(x) = x^T A x$ into scalar algebra to see what happens:

$$f(x) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1^2 + 3x_1 x_2 + 2x_2^2$$

Taking partial derivatives with respect to x_1 and x_2:

$$\nabla_x f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 + 4x_2 \end{bmatrix}$$

Step 2 — Compare $(A + A^T)x$ vs. $2Ax$:

- Using the correct formula $(A + A^T)x$:

$$(A + A^T)x = \left(\begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 3 & 2 \end{bmatrix} \right) \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 + 4x_2 \end{bmatrix} \checkmark$$

- Whereas $2Ax = \begin{bmatrix} 2x_1 + 6x_2 \\ 4x_2 \end{bmatrix}$ is completely wrong because $x_1 x_2$ affects *both* $\frac{\partial f}{\partial x_1}$ and $\frac{\partial f}{\partial x_2}$!

🎯 Exam Takeaway & Fast Shortcuts

- Always check whether the matrix A in $x^T A x$ is specified as **symmetric** ($A = A^T$). If the question says "*For an arbitrary square matrix $A \in \mathbb{R}^{n \times n}$*", the gradient is $(A + A^T)x$ and the Hessian is $A + A^T$!

2.2 🌟 [Core — Bread & Butter] Canonical Deep Learning Backprop Derivations (Softmax + CE & Linear Layers)

💡 Intuition (Plain English First)

Two backpropagation gradients show up on almost every Deep Learning assessment:

1. **Softmax + Cross-Entropy Loss ($\frac{\partial \mathcal{L}}{\partial z} = p - y$):** Suppose a classifier outputs raw logits $z = [z_1, \ldots, z_K]^T$, converts them into probabilities $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ via Softmax, and computes Cross-Entropy loss $\mathcal{L} = -\sum_k y_k \log p_k$

against a one-hot target y. Individually, the derivative of Softmax is a messy matrix ($p_k(\delta_{ik} - p_i)$) and the derivative of $-\log p_k$ has a $1/p_k$ that can blow up. Yet when you combine them via the Chain Rule, **the p_k in the numerator and denominator cancel out cleanly**, leaving the most elegant gradient in all of Deep Learning:

$$\nabla_z \mathcal{L} = p - y \quad \text{(Predicted Probability} - \text{True Label!)}$$

2. **Linear Layer Backprop via "Dimension Matching":** In a linear layer $Y = XW + \mathbf{1}b^T$, you receive the upstream gradient $G = \frac{\partial \mathcal{L}}{\partial Y}$ from the layer above and need to find $\frac{\partial \mathcal{L}}{\partial W}$ and $\frac{\partial \mathcal{L}}{\partial X}$. You **never** need to take elementwise derivatives—because matrix multiplication is linear, $\frac{\partial \mathcal{L}}{\partial W}$ must be a product of X and G, and there is **only one way to transpose and multiply them so the matrix dimensions ($n \times m$) match the shape of W!**

📐 The Math & Every Symbol Explained

1. Softmax Jacobian & Cross-Entropy Gradient

Let $z \in \mathbb{R}^K$ be the raw logit vector, $p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ be the Softmax probabilities, and $y \in \mathbb{R}^K$ be any target distribution with $\sum_{k=1}^K y_k = 1$:

- **Softmax Jacobian ($\frac{\partial p_k}{\partial z_i}$):**

$$\frac{\partial p_k}{\partial z_i} = p_k(\delta_{ik} - p_i) = \begin{cases} p_i(1 - p_i) & \text{if } i = k \\ -p_i p_k & \text{if } i \neq k \end{cases}$$

- **Chain Rule with Cross-Entropy $\mathcal{L} = -\sum_{k=1}^K y_k \log p_k$:**

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial p_k}\frac{\partial p_k}{\partial z_i} = -\sum_{k=1}^K \left(\frac{y_k}{p_k}\right) p_k(\delta_{ik} - p_i) = -\sum_{k=1}^K y_k\delta_{ik} + p_i \underbrace{\sum_{k=1}^K y_k}_{1} = \mathbf{p_i} - \mathbf{y_i}$$

- **Numerical Stability: The Log-Sum-Exp Shift Trick (Crucial for FP16/BF16 Hardware!):** If a logit is $z_i = 100$, $e^{100} \approx 2.7 \times 10^{43}$ overflows FP16 (max value $65,504 \to \text{NaN}$). If all logits are $z_i = -100$, all $e^{-100} \to 0$, giving $0/0 = \text{NaN}$. By subtracting the maximum logit $m = \max_k z_k$ from every logit before exponentiating:

$$p_i = \frac{e^{z_i - m}}{\sum_{k=1}^K e^{z_k - m}}, \qquad \log \sum_{k=1}^K e^{z_k} = m + \log \sum_{k=1}^K e^{z_k - m}$$

  The probabilities p_i are **100% mathematically identical** (since e^{-m} cancels from top and bottom), and the largest exponent is guaranteed to be $e^0 = 1$—preventing both overflow and underflow!

2. Linear Layer Backpropagation ($Y = XW + \mathbf{1}b^T$)

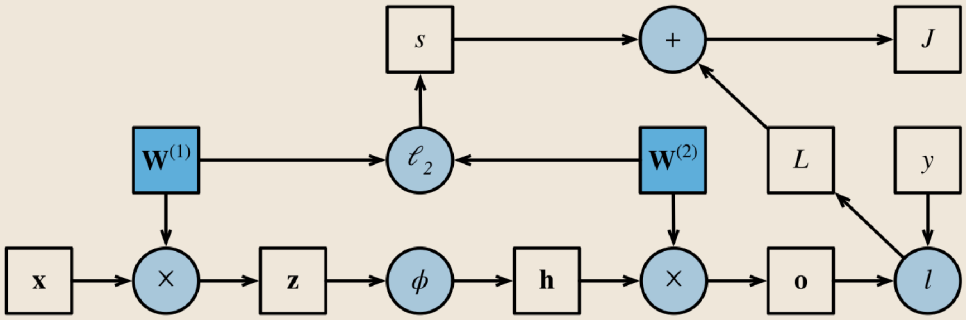
Let's write down the exact shape (n rows, m columns) of every tensor:

- $X \in \mathbb{R}^{N \times d_{\text{in}}}$**:** Input batch ($N$ sample rows, d_{in} input feature columns).
- $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$**:** Weight matrix ($d_{\text{in}}$ rows, d_{out} columns).

- $b \in \mathbb{R}^{d_{\text{out}}}$: Bias vector (broadcast across all N rows via $\mathbf{1} \in \mathbb{R}^N$).
- $Y \in \mathbb{R}^{N \times d_{\text{out}}}$: Output activations, and $G = \frac{\partial \mathcal{L}}{\partial Y} \in \mathbb{R}^{N \times d_{\text{out}}}$: Upstream gradient.

Now match shapes to get the three gradients:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial W}}_{(d_{\text{in}} \times d_{\text{out}})} = \underbrace{X^T}_{(d_{\text{in}} \times N)} \underbrace{G}_{(N \times d_{\text{out}})}, \qquad \underbrace{\frac{\partial \mathcal{L}}{\partial X}}_{(N \times d_{\text{in}})} = \underbrace{G}_{(N \times d_{\text{out}})} \underbrace{W^T}_{(d_{\text{out}} \times d_{\text{in}})}, \qquad \underbrace{\frac{\partial \mathcal{L}}{\partial b}}_{(d_{\text{out}} \times 1)} = \underbrace{G^T}_{(d_{\text{out}} \times N)} \underbrace{\mathbf{1}}_{(N \times 1)}$$



Computational Graph of Forward Pass & Backpropagation with L2 Weight Decay (D2L)

🔢 Step-by-Step Toy Example

Problem: A 3-class classifier outputs logits $z = [0,\ \ln 2,\ \ln 5]^T$ for a sample whose true class is Class 2 ($y = [0, 1, 0]^T$). Compute the exact gradient $\nabla_z \mathcal{L}$ of the Cross-Entropy loss with respect to the logits z.

Step 1 — Compute Softmax probabilities p:

- Exponentiate each logit: $e^0 = 1, \quad e^{\ln 2} = 2, \quad e^{\ln 5} = 5.$
- Sum of exponentials: $1 + 2 + 5 = 8.$
- Therefore:

$$p = \begin{bmatrix} 1/8 \\ 2/8 \\ 5/8 \end{bmatrix} = \begin{bmatrix} 0.125 \\ 0.250 \\ 0.625 \end{bmatrix}$$

Step 2 — Apply $\nabla_z \mathcal{L} = p - y$:

$$\nabla_z \mathcal{L} = p - y = \begin{bmatrix} 0.125 \\ 0.250 \\ 0.625 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} +0.125 \\ -0.750 \\ +0.625 \end{bmatrix}$$

- *(Intuition check: Because Gradient Descent subtracts $\eta \nabla_z \mathcal{L}$, the negative gradient -0.750 on Class 2 will **increase** z_2, while the positive gradients $+0.125$ and $+0.625$ will **push down** the wrong logits z_1 and z_3! Also notice that the sum of the gradient entries is always $\sum_i (p_i - y_i) = 1 - 1 = 0$.)*

🎯 Exam Takeaway & Fast Shortcuts

- **Zero-Sum Gradient Invariant:** Because $\sum_i p_i = 1$ and $\sum_i y_i = 1$, the entries of the Softmax + Cross-Entropy logit gradient $\nabla_z \mathcal{L} = p - y$ **always sum to** 0! On an MCQ, you can immediately eliminate any gradient vector

whose entries do not sum to zero.

2.3 🌟 [Core — Bread & Butter] Convexity, Gradient Descent Step-Size Limits & Newton's Method

💡 Intuition (Plain English First)

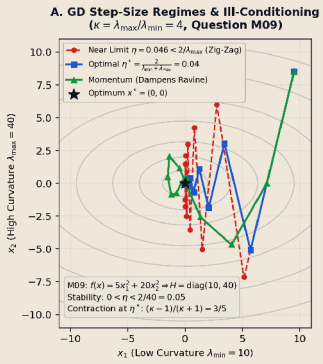
Why do we care if a loss function $f(x)$ is **convex** (bowl-shaped)? Because on a convex surface, **every local minimum is guaranteed to be a global minimum**, and standing somewhere with zero gradient ($\nabla f(x^*) = 0$) guarantees you are at the bottom of the bowl.

Now imagine rolling a ball down a quadratic bowl $f(x) = \frac{1}{2}x^T Q x - b^T x$ using Gradient Descent ($x_{t+1} = x_t - \eta \nabla f(x_t)$):

- What happens if your learning rate η is **too big**? Instead of stepping toward the bottom of the valley, you **jump completely across the valley** to a point higher up on the opposite wall! On the next step, the steeper slope launches you even higher back across, and your loss explodes to ∞.
- Which direction of the bowl determines the maximum safe learning rate $\eta_{\max}$? The **steepest, narrowest wall** of the bowl—which corresponds to the **largest eigenvalue $\lambda_{\max}$ of the Hessian Q**! Specifically, you must keep $\eta < \frac{2}{\lambda_{\max}}$ or Gradient Descent will diverge.

What if the bowl is a **narrow canyon/ravine** where the side walls are super steep ($\lambda_{\max} = 100$) while the floor slopes very gently toward the exit ($\lambda_{\min} = 1$)?

- Because of the steep canyon walls, you are forced to use a tiny learning rate $\eta < \frac{2}{100} = 0.02$ so you don't bounce off the walls.
- But along the flat canyon floor ($\lambda_{\min} = 1$), a step size of 0.02 barely moves! This ratio $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the **condition number**: when $\kappa \gg 1$, vanilla Gradient Descent zig-zags across the steep walls while crawling slowly along the floor.
- **Momentum** fixes this by averaging velocity over time: the back-and-forth zig-zags across the steep walls (+ and - signs) **cancel each other out**, while the consistent steps along the canyon floor (all same sign) **accumulate and accelerate**!
- **Newton's Method** goes even further: it multiplies the gradient by the **inverse Hessian** $[\nabla^2 f(x)]^{-1}$, which stretches the narrow canyon back into a perfectly round circular bowl ($\kappa = 1$) and jumps to the exact bottom of any quadratic bowl in **1 single step**! (Its catch: storing and inverting an $n \times n$ Hessian for a neural network with $n = 10^9$ weights costs $\mathcal{O}(n^2)$ memory and $\mathcal{O}(n^3)$ compute, which is why Deep Learning uses diagonal approximations like Adam instead).

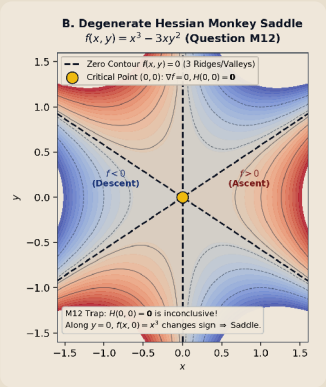


Gradient Descent Step-Size Regimes on a 2D Quadratic Bowl

▼ 🔬 Optional Deep-Dive: Degenerate Critical Points & The Monkey Saddle ($\nabla^2 f = 0$) (Click to Expand)

When $\nabla f(x^*) = 0$, the Hessian $\nabla^2 f(x^*)$ classifies the critical point:

- All $\lambda_i > 0 \implies$ **Strict Local Minimum**; all $\lambda_i < 0 \implies$ **Strict Local Maximum**; mixed signs ($\lambda_i > 0$ and $\lambda_j < 0$) $\implies$ **Saddle Point**.
- **Inconclusive / Degenerate Case ($\det(\nabla^2 f) = 0$):** If some $\lambda_i = 0$ and the rest have the same sign (or all $\lambda_i = 0$, as in the **Monkey Saddle** $f(x, y) = x^3 - 3xy^2$ at $(0, 0)$), the 2nd-order Taylor test is **inconclusive**—you must inspect higher-order terms!



Degenerate Critical Point: Monkey Saddle Surface z = x^3 - 3xy^2

📐 The Math & Every Symbol Explained

1. Rules for Spotting Convex Functions ($f : \mathbb{R}^n \to \mathbb{R}$)

- **Twice-Differentiable Test:** f is convex $\iff$ its Hessian is Positive Semi-Definite ($\nabla^2 f(x) \succeq 0$) everywhere. f is **strictly convex** (unique global minimum!) if $\nabla^2 f(x) \succ 0$ everywhere.
- **3 Operations That Always Preserve Convexity:**
  1. **Non-negative weighted sums:** $\sum_i \alpha_i f_i(x)$ with $\alpha_i \geq 0$.
  2. **Affine composition:** $g(x) = f(Ax + b)$ is convex whenever f is convex.
  3. **Pointwise Maximum ($\max_i f_i(x)$):** The maximum of *any* collection of convex/linear functions is **always convex**! *(This immediately proves that **ReLU** $\max(0, x)$, **SVM Hinge Loss** $\max(0, 1 - yw^T x)$, L_1 **norm** $\sum_i \max(x_i, -x_i)$, and L_∞ **norm** $\max_i |x_i|$ are all convex!)*

2. Gradient Descent Stability on a Quadratic $f(x) = \frac{1}{2}x^TQx - b^Tx$ ($Q \succ 0$)

- Here $\nabla f(x) = Qx - b$ and Hessian $\nabla^2 f(x) = Q$ has eigenvalues $0 < \lambda_{\min} \leq \cdots \leq \lambda_{\max}$.
- One step of Gradient Descent $x_{t+1} = x_t - \eta(Qx_t - b)$ shrinks the error vector $e_t = x_t - x^*$ by:

$$e_{t+1} = (I - \eta Q)e_t$$

- Along the i-th eigenvector of Q, the error is multiplied by $(1 - \eta\lambda_i)$ at every step:
  - $0 < \eta \leq \frac{1}{\lambda_{\max}}$: Monotonic convergence (no overshooting).
  - $\frac{1}{\lambda_{\max}} < \eta < \frac{2}{\lambda_{\max}}$: Damped oscillations (jumps past x^* each step, but gets closer: $-1 < 1 - \eta\lambda_{\max} < 0$).
  - $\eta \geq \frac{2}{\lambda_{\max}}$: **Divergence!** ($|1 - \eta\lambda_{\max}| \geq 1$).

$$\text{Mandatory Stability Condition:} \quad \mathbf{0 < \eta < \frac{2}{\lambda_{\max}}}$$

3. Why L_1 (Lasso) Creates Exact Zeros While L_2 (Ridge) Only Shrinks

- L_2 **Penalty ($\frac{\lambda}{2}w^2$):** Derivative is λw, which shrinks toward 0 as $w \to 0$. So L_2 slows down as it approaches zero and never locks weights to exact 0.
- L_1 **Penalty ($\lambda|w|$):** Has a sharp V-shaped corner at $w = 0$ with constant slope $+\lambda$ on the right and $-\lambda$ on the left. Unless the loss gradient $|\nabla f_0(0)|$ pulls harder than the threshold λ, the weight gets trapped in the V-corner at **exact** $w^* = 0$!

▼ 📌 **Optional Deep-Dive / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact Optimal Step Size & Formal Subdifferential ∂|0| = [-1, 1]**

- **Optimal Constant Learning Rate:** To minimize $\max_i |1 - \eta\lambda_i|$, make the slowest and fastest modes have equal magnitude: $1 - \eta^*\lambda_{\min} = -(1 - \eta^*\lambda_{\max}) \implies \eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$, which gives per-step contraction factor $\frac{\kappa - 1}{\kappa + 1}$ where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$.
- **Formal Subdifferential of $|w|$:** At a non-differentiable corner, the subdifferential $\partial f(w)$ is the set of slopes of all lines tangent below f at w. For $|w|$, $\partial|w| = \{\text{sign}(w)\}$ for $w \neq 0$, and at $w = 0$ any slope in $[-1, 1]$ stays below $|w|$, so $\partial|0| = [-1, 1]$. Zero is in the subdifferential of $f_0(w) + \lambda|w|$ at $w = 0 \iff 0 \in \nabla f_0(0) + \lambda[-1, 1] \iff |\nabla f_0(0)| \leq \lambda$.

🔢 Step-by-Step Toy Example

Problem: Consider minimizing $f(x_1, x_2) = 5x_1^2 + 50x_2^2$ using Gradient Descent $x_{t+1} = x_t - \eta\nabla f(x_t)$.

1. Find the Hessian matrix $Q = \nabla^2 f(x)$ and its eigenvalues $\lambda_{\min}, \lambda_{\max}$ and condition number κ.
2. Find the maximum learning rate $\eta_{\max}$ before Gradient Descent diverges.

Step 1 — Compute Gradient & Hessian Q:

- $\nabla f(x) = \begin{bmatrix} 10x_1 \\ 100x_2 \end{bmatrix}$.
- Taking second derivatives gives the diagonal Hessian:

$$Q = \nabla^2 f(x) = \begin{bmatrix} 10 & 0 \\ 0 & 100 \end{bmatrix}$$

(Watch out: because $f(x) = \frac{1}{2}x^T Q x$, a term ax_i^2 has second derivative $2a$, so $5x_1^2 \implies \lambda_1 = 10$ and $50x_2^2 \implies \lambda_2 = 100$!)

- Condition number: $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{100}{10} = \mathbf{10}$.

Step 2 — Maximum Safe Learning Rate $\eta_{\max}$:

$$\eta < \frac{2}{\lambda_{\max}} = \frac{2}{100} = \mathbf{0.02}$$

- Let's verify why $\eta = 0.02$ is the exact tipping point: along x_2, the update is $x_2^{(t+1)} = x_2^{(t)} - \eta(100x_2^{(t)}) = (1 - 100\eta)x_2^{(t)}$. If $\eta = 0.02$, then $1 - 100(0.02) = -1$, so x_2 bounces endlessly between $+x_2^{(0)}$ and $-x_2^{(0)}$!

🎯 Exam Takeaway & Fast Shortcuts

- **The $\frac{1}{2}$ Coefficient Trap:** When given a quadratic $f(x_1, x_2) = ax_1^2 + bx_2^2$ without a $\frac{1}{2}$ in front, remember that the Hessian eigenvalues are $2a$ **and** $2b$ (not a and b!). Therefore, $\eta_{\max} = \frac{2}{\lambda_{\max}} = \frac{2}{2\max(a,b)} = \frac{1}{\max(a,b)}$.

2.4 🔬 [Advanced / Deep-Dive] Constrained Optimization: Lagrange Multipliers & KKT Conditions

💡 Intuition (Plain English First)

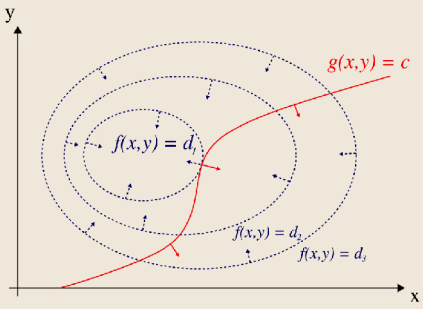
What if you want to minimize a function $f(x)$, but you aren't allowed to go anywhere you want—you must stay inside a fenced region defined by an **inequality constraint** $g(x) \leq 0$ (and/or on a curve defined by an **equality constraint** $h(x) = 0$)?

Let's visualize the two things that can happen when you minimize $f(x)$ subject to $g(x) \leq 0$:

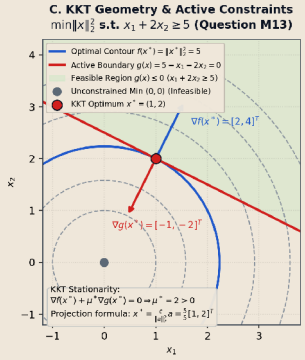
- **Case 1 — The Unconstrained Minimum is Already Inside the Fence ($g(x^*) < 0$, "Inactive Constraint"):** If the bottom of the bowl $f(x)$ is already safely inside the allowed region, **the fence doesn't do anything**! At the optimum x^*, $\nabla f(x^*) = 0$, and the Lagrange multiplier (which measures how hard the fence has to push back against you) is $\mu^* = 0$.
- **Case 2 — The Unconstrained Minimum is Outside the Fence ($g(x^*) = 0$, "Active Constraint"):** You roll downhill until you **slam against the boundary wall** $g(x^*) = 0$. Why can't you slide any lower along the wall? Because at the optimal point x^* on the wall, the direction of steepest descent $-\nabla f(x^*)$ points **straight into the wall**, parallel to the outward normal vector $\nabla g(x^*)$! That means $-\nabla f(x^*) = \mu^* \nabla g(x^*) \implies \nabla f(x^*) + \mu^* \nabla g(x^*) = 0$ with a positive push-back force $\mu^* > 0$.

Notice that in **both** cases:

- Either $g(x^*) < 0$ and $\mu^* = 0$ (inside the fence, zero push-back force),
- Or $g(x^*) = 0$ and $\mu^* \geq 0$ (on the fence wall, positive push-back force). In either case, their product is **always zero**: $\mu^* g(x^*) = 0$. That single equation is called **Complementary Slackness**, and it is the exact mathematical reason why **Support Vector Machines (SVMs) are sparse**!



Lagrange Multiplier Tangency: Gradient Alignment $\nabla f = \lambda \nabla g$ on Constraint Curve



KKT Complementary Slackness: Inactive Interior vs. Active Boundary Optimum

▼ 🔬 Optional Deep-Dive: The 4 Formal KKT Equations & Worked Boundary Example (Click to Expand)

📐 The Math & Every Symbol Explained

For the constrained optimization problem:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_i(x) \leq 0 \;\; (i = 1, \ldots, m), \quad h_j(x) = 0 \;\; (j = 1, \ldots, p)$$

We form the **Lagrangian function**:

$$\mathcal{L}(x, \mu, \lambda) = f(x) + \sum_{i=1}^{m} \mu_i g_i(x) + \sum_{j=1}^{p} \lambda_j h_j(x)$$

- μ_i **(KKT / Inequality Multipliers):** Penalize violating the boundary walls $g_i(x) \leq 0$.
- λ_j **(Lagrange / Equality Multipliers):** Enforce staying on the curves $h_j(x) = 0$ (can be positive, negative, or zero).

The 4 Karush-Kuhn-Tucker (KKT) Optimality Conditions

At an optimal solution (x^*, μ^*, λ^*):

1. **Stationarity (Forces Balance):** $\nabla_x \mathcal{L}(x^*, \mu^*, \lambda^*) = \nabla f(x^*) + \sum_i \mu_i^* \nabla g_i(x^*) + \sum_j \lambda_j^* \nabla h_j(x^*) = 0$.
2. **Primal Feasibility (Obeys Constraints):** $g_i(x^*) \leq 0$ and $h_j(x^*) = 0$.
3. **Dual Feasibility (One-Way Fence):** $\mu_i^* \geq 0$ for all inequality constraints! (Equality multipliers λ_j^* have no sign restriction).
4. **Complementary Slackness:** $\mu_i^* g_i(x^*) = 0$ for every i.
    - **Connection to SVMs:** In an SVM, each training point x_i gives a margin constraint $g_i(w) = 1 - y_i(w^T x_i + b) \leq 0$ with multiplier $\alpha_i^* \geq 0$. By Complementary Slackness $\alpha_i^* \big(1 - y_i(w^T x_i + b)\big) = 0$:
        - Any training point safely outside the margin ($y_i(w^T x_i + b) > 1 \implies g_i(w) < 0$) is **forced to have** $\alpha_i^* = 0$ (it plays zero role in the final classifier $w^* = \sum_i \alpha_i^* y_i x_i$!).
        - Only the **Support Vectors** sitting on (or inside, in Soft-Margin SVM) the margin have $\alpha_i^* > 0$.

🔢 Step-by-Step Toy Example

Problem: Minimize $f(x_1, x_2) = (x_1 - 3)^2 + (x_2 - 4)^2$ (squared distance to the point $(3, 4)$) subject to staying inside the disk $x_1^2 + x_2^2 \leq 4$ (i.e., $g(x_1, x_2) = x_1^2 + x_2^2 - 4 \leq 0$).

Step 1 — Check Case 1 (Is the unconstrained minimum inside the fence?):

- The unconstrained minimum of f is $(3, 4)$.
- Plug $(3, 4)$ into g: $3^2 + 4^2 - 4 = 25 - 4 = 21 > 0$ (outside the disk of radius 2!).
- Therefore, the constraint **must be active** ($g(x^*) = 0$, **so** $x_1^2 + x_2^2 = 4$) and $\mu^* > 0$.

Step 2 — Use Stationarity ($\nabla f(x^*) + \mu^* \nabla g(x^*) = 0$):

- Geometrically, the closest point on the circle of radius 2 to $(3, 4)$ (which is at distance $\sqrt{3^2 + 4^2} = 5$ from the origin) lies along the straight ray from $(0, 0)$ to $(3, 4)$:

$$x^* = \frac{2}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 1.6 \end{bmatrix}$$

- Check $\nabla f(x^*) + \mu^* \nabla g(x^*) = 0$ at $x^* = (1.2, 1.6)$:

$$\nabla f(x^*) = 2\begin{bmatrix}1.2-3\\1.6-4\end{bmatrix} = \begin{bmatrix}-3.6\\-4.8\end{bmatrix}, \qquad \nabla g(x^*) = 2\begin{bmatrix}1.2\\1.6\end{bmatrix} = \begin{bmatrix}2.4\\3.2\end{bmatrix}$$

$$\begin{bmatrix}-3.6\\-4.8\end{bmatrix} + \mu^* \begin{bmatrix}2.4\\3.2\end{bmatrix} = 0 \implies \mu^* = \frac{3.6}{2.4} = \mathbf{1.5} > \mathbf{0}\ \checkmark$$

🎯 Exam Takeaway & Fast Shortcuts

- Always check the **unconstrained optimum first** on any KKT problem! If the unconstrained minimum already satisfies $g(x) \le 0$, you are done in 5 seconds ($\mu^* = 0$). If it violates $g(x) \le 0$ (for a convex problem), you immediately know the solution lies **on the boundary** $g(x^*) = 0$.

Day 3: Probability, Statistics & Information Theory

↑ Mục Lục

3.1 🌟 [Core — Bread & Butter] Conditional Probability, Bayes' Theorem & The Odds-Ratio Speed Hack

💡 Intuition (Plain English First)

Why do even smart engineers get tricked by Bayes' Theorem questions (like medical test or spam filter false-positive problems)? Because human intuition forgets the **Base Rate (Prior)**: if a disease only affects 1 out of 100 people (1% prevalence), and a test has a 9% false-positive rate, then out of 100 people:

- Only 1 **person** actually has the disease (and tests positive).
- Out of the 99 **healthy people**, 9% of them (≈ 9 people) *also* get a false positive! So out of the $1 + 9 = 10$ people who got a positive test result, only 1 **out of** 10 **(10%)** is actually sick!

Instead of plugging decimals into $P(D \mid +) = \frac{P(+|D)P(D)}{P(+|D)P(D)+P(+|\neg D)P(\neg D)}$ under time pressure, use the **Odds-Ratio Form of Bayes' Rule**, which turns messy decimal division into a 10-second integer multiplication.

📐 The Math & Every Symbol Explained

1. Standard Bayes' Rule & Law of Total Probability

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)} = \frac{P(B \mid A)P(A)}{P(B \mid A)P(A) + P(B \mid A^c)P(A^c)}$$

- $P(A)$ **(Prior):** Probability of A before seeing evidence B.
- $P(B \mid A)$ **(Likelihood / Sensitivity / True Positive Rate):** Probability of observing evidence B when A is true.
- $P(B \mid A^c)$ **(False Positive Rate** $= 1 -$ Specificity**):** Probability of observing evidence B when A is false.

- $P(A \mid B)$ **(Posterior):** Updated probability that A is true after observing B.

2. The 10-Second Odds-Ratio Speed Hack

Divide $P(D \mid +)$ by $P(\neg D \mid +)$ and the denominator $P(+)$ cancels out completely:

$$\underbrace{\frac{P(D \mid +)}{P(\neg D \mid +)}}_{\text{Posterior Odds}} = \underbrace{\frac{P(+ \mid D)}{P(+ \mid \neg D)}}_{\text{Likelihood Ratio}} \times \underbrace{\frac{P(D)}{P(\neg D)}}_{\text{Prior Odds}}$$

Once you have the Posterior Odds as a ratio $a : b$, the probability is simply $P(D \mid +) = \frac{a}{a+b}$!

🔢 Step-by-Step Toy Example

Problem: A defective chip occurs in 1% of factory batches ($P(D) = 0.01$). An automated camera detects a defective batch 99% of the time ($P(+ \mid D) = 0.99$), and triggers a false alarm on a good batch 9% of the time ($P(+ \mid \neg D) = 0.09$). If the alarm goes off, what is $P(D \mid +)$?

Step 1 — Write down Prior Odds and Likelihood Ratio:

- Prior Odds: $\frac{P(D)}{P(\neg D)} = \frac{0.01}{0.99} = \frac{1}{99}$.
- Likelihood Ratio: $\frac{P(+|D)}{P(+|\neg D)} = \frac{0.99}{0.09} = 11$.

Step 2 — Multiply to get Posterior Odds:

$$\text{Posterior Odds} = 11 \times \frac{1}{99} = \frac{11}{99} = \frac{1}{9} \quad (1:9)$$

Step 3 — Convert Odds $1:9$ to Probability:

$$P(D \mid +) = \frac{1}{1+9} = \frac{1}{10} = \mathbf{10\%}$$

🎯 Exam Takeaway & Fast Shortcuts

- Watch out for wording: "**95% Specificity**" means True Negative Rate $P(- \mid \neg D) = 0.95$, which means the **False Positive Rate is** $P(+ \mid \neg D) = 1 - 0.95 = 0.05$!

3.2 🌟 [Core — Bread & Butter] Core Probability Distributions & Their ML Roles

💡 Intuition (Plain English First)

Each standard probability distribution models a specific physical process in Machine Learning:

- **Bernoulli & Binomial:** Flipping a biased coin once (Bernoulli(p)) or n times (Binomial(n, p))—used in binary classification and **Dropout masks**.
- **Poisson(λ) & Exponential(λ):** Imagine packets arriving at an NPU at an average rate of λ packets per millisecond:

- **Poisson(λ)** counts the *discrete number of packets* arriving in 1 ms (uniquely, its **Mean equals its Variance** $= \lambda$!).
- **Exponential(λ)** measures the *continuous waiting time* until the next packet arrives. It is the **only continuous memoryless distribution**: $P(T > s + t \mid T > s) = P(T > t)$ (having waited s seconds already gives you zero advantage—the expected remaining wait time is still $1/\lambda$!).

- **Gaussian $\mathcal{N}(\mu, \sigma^2)$ vs. Laplace(μ, b):**
  - **Gaussian** has a squared penalty $-\frac{(x-\mu)^2}{2\sigma^2}$ in the exponent—putting a Gaussian prior on weights w creates L_2 **(Ridge) regularization**. Among all continuous distributions on $\mathbb{R}$ with fixed variance σ^2, the Gaussian has the **maximum possible Shannon differential entropy**.
  - **Laplace** has an absolute-value penalty $-\frac{|x-\mu|}{b}$ in the exponent (a sharp tent peak at μ with heavier tails)—putting a Laplace prior on weights w creates L_1 **(Lasso) sparsity**!

📐 The Math & Reference Table

Distribution	PMF / PDF	Mean $\mathbb{E}[X]$	Variance $\mathrm{Var}(X)$	Key ML / Exam Property
Bernoulli(p)	$p^x(1-p)^{1-x}$ $(x \in \{0,1\})$	p	$p(1-p)$	Max variance $= 0.25$ at $p = 0.5$
Binomial(n, p)	$\binom{n}{k}p^k(1-p)^{n-k}$	np	$np(1-p)$	Sum of n i.i.d. Bernoulli(p) trials
Poisson(λ)	$\frac{\lambda^k e^{-\lambda}}{k!}$ $(k = 0, 1, 2, \dots)$	λ	λ	**Mean = Variance** $= \lambda$; sum $\mathrm{Pois}(\lambda_1) + \mathrm{Pois}(\lambda_2) = \mathrm{Pois}(\lambda_1 + \lambda_2)$
Uniform(a, b)	$\frac{1}{b-a}$ $(a \le x \le b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	Used in Xavier/He Uniform weight init ($\pm\sqrt{3 \cdot \mathrm{Var}}$)
Exponential(λ)	$\lambda e^{-\lambda x}$ $(x \ge 0)$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	**Memoryless:** $P(X > s + t \mid X > s) = P(X > t) = e^{-\lambda t}$
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2	Prior $\Longrightarrow$ L_2 **(Ridge)**; Likelihood $\Longrightarrow$ **MSE loss**
Laplace(μ, b)	$\frac{1}{2b}e^{-\frac{\lvert x-\mu\rvert}{b}}$	μ	$2b^2$	Prior $\Longrightarrow$ L_1 **(Lasso)**; Likelihood $\Longrightarrow$ **MAE (L_1) loss**

🔢 Step-by-Step Toy Example

Problem: You want to initialize a neural network weight $W \sim \mathrm{Uniform}(-a, a)$ so that its mean is 0 and its variance is $\mathrm{Var}(W) = \sigma^2$. Solve for the bound a in terms of σ.

Step 1 — Plug $[-a, a]$ into the Uniform variance formula $\frac{(b-a)^2}{12}$:

$$\mathrm{Var}(W) = \frac{(a - (-a))^2}{12} = \frac{(2a)^2}{12} = \frac{4a^2}{12} = \frac{a^2}{3}$$

Step 2 — Set $\frac{a^2}{3} = \sigma^2$ and solve for a:

$$\frac{a^2}{3} = \sigma^2 \implies a^2 = 3\sigma^2 \implies \mathbf{a = \sqrt{3}\,\sigma}$$

(This is why Xavier and Kaiming/He Uniform initializers always have an extra $\sqrt{3}$ inside the square root compared to their Normal counterparts!)

🎯 Exam Takeaway & Fast Shortcuts

- Memorize the 12 **in the denominator** of Uniform variance $\frac{(b-a)^2}{12}$ and the $2b^2$ variance of Laplace(μ, b)—both are frequent 30-second calculation questions.

3.3 🌟 [Core — Bread & Butter] Expectation, Variance, Correlated Ensembles & Probability Inequalities

💡 Intuition (Plain English First)

Three rules about Expectation and Variance govern almost every statistical derivation in ML:

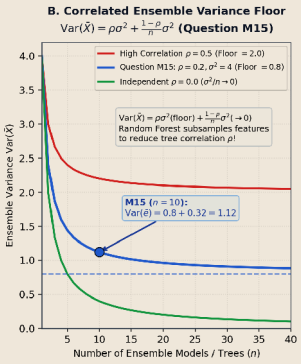
1. **Linearity of Expectation ($\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$) Is Magic:** It holds **even when X and Y are completely dependent!** Whether X and Y are independent coin flips or 100% correlated variables, the average of a sum is *always* the sum of the averages.
2. **Variance Squares Constants—So Variances ADD Even When Subtracting!** Because variance measures squared spread $\mathbb{E}[(X-\mu)^2]$, multiplying X by a constant a scales variance by a^2: $\text{Var}(aX) = a^2\text{Var}(X)$. What happens if $a = -1$? $(-1)^2 = +1$! So if X and Y are independent, subtracting two noisy measurements $X - Y$ does **not** cancel their noise—it **adds** their variances:

   $$\text{Var}(X - Y) = (1)^2\text{Var}(X) + (-1)^2\text{Var}(Y) = \text{Var}(X) + \text{Var}(Y)$$

3. **Why Random Forests Subsample Features (Averaging Correlated Trees):** If you average n *independent* models each with variance σ^2, the ensemble variance drops to $\frac{\sigma^2}{n}$, which goes to 0 as $n \to \infty$. What if the n models (like decision trees trained on bootstrapped copies of the same dataset) have pairwise correlation $\rho > 0$? Then averaging them gives:

   $$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$$

   Notice that as you add infinitely many trees ($n \to \infty$), the second term $\frac{1-\rho}{n}\sigma^2$ goes to 0, **but the first term $\rho\sigma^2$ never goes away**! That irreducible floor $\rho\sigma^2$ is why **Random Forest** randomly restricts each split to a subset of $m \approx \sqrt{d}$ features—to de-correlate the trees (reduce ρ) and lower the variance floor!



Ensemble Variance Reduction & Correlated Tree Floor ρσ²

📐 The Math & Every Symbol Explained

1. Expectation & Variance of Linear Combinations

$$\mathbb{E}\left[\sum_{i=1}^{n} a_i X_i\right] = \sum_{i=1}^{n} a_i \mathbb{E}[X_i] \quad \text{(Holds ALWAYS, even if dependent!)}$$

$$\text{Var}\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i^2 \text{Var}(X_i) + 2 \sum_{1 \le i < j \le n} a_i a_j \text{Cov}(X_i, X_j)$$

- **Covariance:** $\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$.
- **Pearson Correlation:** $\rho(X, Y) = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.
- **Independence vs. Zero Correlation Trap:**

  $$X \perp Y \implies \text{Cov}(X, Y) = 0, \qquad \text{but} \qquad \text{Cov}(X, Y) = 0 \not\implies X \perp Y!$$

  *(Classic Counterexample: Let $X \sim \mathcal{N}(0, 1)$ or $X \in \{-1, 0, 1\}$ symmetrically, and let $Y = X^2$. Clearly Y is 100% determined by X, yet $\text{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0 - 0 = 0$ because correlation only measures **linear** relationships!)*

▼ 🔬 **Optional Deep-Dive: Law of Total Variance (Eve's Law) & Jensen / Markov / Chebyshev Inequalities (Click to Expand)**

2. Law of Total Expectation & Law of Total Variance (Eve's Law)

$$\mathbb{E}[X] = \mathbb{E}_Y\big[\mathbb{E}[X \mid Y]\big], \qquad \mathrm{Var}(X) = \mathbb{E}_Y\big[\mathrm{Var}(X \mid Y)\big] + \mathrm{Var}_Y\big(\mathbb{E}[X \mid Y]\big)$$

3. The 3 Essential Probability Inequalities

1. **Jensen's Inequality:**
   - If g is **convex** (e.g., $g(x) = x^2$ or e^x): $\mathbb{E}[g(X)] \geq g(\mathbb{E}[X])$. *(Notice $\mathbb{E}[X^2] \geq (\mathbb{E}[X])^2$ immediately proves $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \geq 0$!)*
   - If g is **concave** (e.g., $g(x) = \log x$): $\mathbb{E}[\log X] \leq \log(\mathbb{E}[X])$. *(This is the 1-line proof that KL Divergence $D_{\mathrm{KL}}(P \parallel Q) \geq 0$ and the VAE/Diffusion ELBO bound!)*
2. **Markov's Inequality (for any non-negative $X \geq 0$ and $a > 0$):**

$$P(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

3. **Chebyshev's Inequality (for any X with mean μ and variance σ^2):**

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$$

🔢 Step-by-Step Toy Example

Problem: An ensemble averages $n = 100$ regression trees. Each tree has prediction variance $\sigma^2 = 10$, and the pairwise correlation between any two trees is $\rho = 0.25$.

1. What is the variance of the 100-tree ensemble average $\bar{X}$?
2. What is the lowest possible variance achievable if you increase the number of trees to $n = \infty$?

Step 1 — Derive or Apply the Correlated Average Formula:

$$\mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\Big[\underbrace{n\sigma^2}_{n \text{ diagonal terms}} + \underbrace{n(n-1)\rho\sigma^2}_{n(n-1) \text{ off-diagonal pairs}}\Big] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$$

Step 2 — Plug in $n = 100, \sigma^2 = 10, \rho = 0.25$:

$$\mathrm{Var}(\bar{X}) = (0.25)(10) + \frac{1 - 0.25}{100}(10) = 2.5 + 0.075 = \mathbf{2.575}$$

Step 3 — Limit as $n \to \infty$:

$$\lim_{n\to\infty} \mathrm{Var}(\bar{X}) = \rho\sigma^2 = (0.25)(10) = \mathbf{2.5}$$

🎯 Exam Takeaway & Fast Shortcuts

- If $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent, then $X - Y \sim \mathcal{N}(\mu_1 - \mu_2,\ \sigma_1^2 + \sigma_2^2)$. Never subtract the variances!

3.4 🌟 [Core — Bread & Butter] Maximum Likelihood (MLE) vs. MAP & Estimator Bias

💡 Intuition (Plain English First)

Suppose you have an unknown parameter θ (like the probability of heads on a coin, or the weights of a neural network) and observed data $D = \{x_1, \ldots, x_n\}$:

- **Maximum Likelihood Estimation (MLE):** Asks "*What value of θ makes the data D we just observed as probable as possible?*"

  $$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} P(D \mid \theta) = \arg\max_{\theta} \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

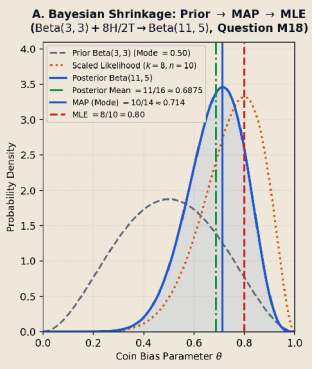
  MLE relies **100% on the observed data**, which can overfit badly when n is tiny (e.g., if you flip a coin $n = 2$ times and get 2 heads, MLE confidently claims the coin has $\hat{\theta}_{\text{MLE}} = 2/2 = 100\%$ chance of heads forever!).
- **Maximum A Posteriori (MAP):** Combines the data likelihood $P(D \mid \theta)$ with a **Prior belief** $P(\theta)$ via Bayes' Rule:

  $$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} P(\theta \mid D) = \arg\max_{\theta} \Big[\log P(D \mid \theta) + \log P(\theta) \Big]$$

  Notice what $\log P(\theta)$ does: it acts as a **regularization penalty** that pulls $\hat{\theta}_{\text{MAP}}$ toward the prior when n is small, and gradually fades away as $n \to \infty$!

What does it mean for an estimator $\hat{\theta}$ to be **biased**?

- **Bias:** $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$ (whether $\hat{\theta}$ systematically undershoots or overshoots θ on average).
- **Why is the Gaussian MLE variance $\hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ biased downward ($\mathbb{E}[\hat{\sigma}^2_{\text{MLE}}] = \frac{n-1}{n}\sigma^2$)?** Because the sample mean $\bar{x}$ is calculated from the *exact same data points* $x_1, \ldots, x_n$, so $\bar{x}$ sits slightly closer to those specific points x_i than the true population mean μ does! Dividing by $n - 1$ instead of n (**Bessel's Correction**) compensates for the 1 degree of freedom used to estimate $\bar{x}$.



Beta-Bernoulli Bayesian Shrinkage: MLE vs. MAP vs. Posterior Mean

---

📐 The Math & Every Symbol Explained

1. Estimator MSE = Variance + Bias²

For any estimator $\hat{\theta}$ of a true parameter θ:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}\big[(\hat{\theta} - \theta)^2\big] = \text{Var}(\hat{\theta}) + \big[\text{Bias}(\hat{\theta})\big]^2$$

2. Gaussian MLE ($\mathcal{N}(\mu, \sigma^2)$) vs. Unbiased Sample Variance

$$\hat{\mu}_{\text{MLE}} = \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \;\; (\text{Unbiased: } \mathbb{E}[\bar{x}] = \mu), \qquad \hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 \;\; \left(\text{Biased: } \mathbb{E}[\hat{\sigma}^2_{\text{MLE}}] = \frac{n-1}{n}\sigma^2\right)$$

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \;\; \big(\text{Unbiased: } \mathbb{E}[S^2] = \sigma^2\big)$$

▼ 🔬 **Optional Deep-Dive: Uniform$(0, \theta)$ Boundary MLE (Click to Expand)**

3. Uniform$(0, \theta)$ MLE (The Classic Boundary Exam Problem)

Suppose $x_1, \ldots, x_n \overset{\text{i.i.d.}}{\sim} \text{Uniform}(0, \theta)$. The density of each point is $\frac{1}{\theta}$ **only if** $\theta \geq x_i$ for every i (i.e., $\theta \geq \max_i x_i$). Thus the likelihood is:

$$L(\theta) = \frac{1}{\theta^n} \cdot \mathbb{I}\big(\theta \geq \max(x_1, \ldots, x_n)\big)$$

- **Why setting $\frac{dL}{d\theta} = 0$ fails:** $\frac{1}{\theta^n}$ is strictly decreasing! To make $\frac{1}{\theta^n}$ as large as possible while still satisfying $\theta \geq \max_i x_i$, we must make θ as small as legally allowed:

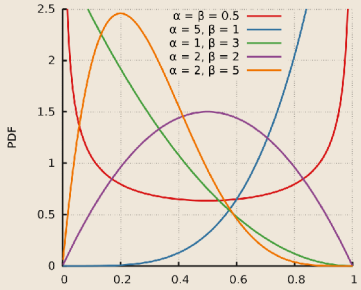
  $$\hat{\theta}_{\text{MLE}} = \max(x_1, \ldots, x_n), \qquad \mathbb{E}[\hat{\theta}_{\text{MLE}}] = \frac{n}{n+1}\theta \;\; (\text{Biased downward!})$$

4. Beta-Bernoulli Conjugate Prior (Coin Flip MAP vs. Posterior Mean)

Suppose a coin has unknown bias $\theta \in [0, 1]$ with a $\text{Beta}(\alpha, \beta)$ prior ($p(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$), and we observe k **heads in** n **flips**:

- **Posterior Distribution:** $\theta \mid D \sim \text{Beta}(\alpha + k,\ \beta + n - k)$. *(Intuition: The prior* $\text{Beta}(\alpha, \beta)$ *acts like starting with* $\alpha - 1$ ***imaginary prior heads*** *and* $\beta - 1$ ***imaginary prior tails****!)*
- **Three Estimators to Know Cold:**

  $$\hat{\theta}_{\text{MLE}} = \frac{k}{n}, \qquad \hat{\theta}_{\text{MAP}} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2} \;\; (\text{Mode}), \qquad \mathbb{E}[\theta \mid D] = \frac{k + \alpha}{n + \alpha + \beta} \;\; (\text{Posterior Mean})$$



Beta(α, β) Conjugate Prior Density Family Across Shape Parameters

---

🔢 Step-by-Step Toy Example

Problem: You place a $\text{Beta}(\alpha = 3, \beta = 3)$ prior on a coin's bias θ. You flip the coin $n = 4$ times and get $k = 4$ heads. Compute $\hat{\theta}_{\text{MLE}}$, $\hat{\theta}_{\text{MAP}}$, and the Posterior Mean $\mathbb{E}[\theta \mid D]$.

Step 1 — MLE (Data Only):

$$\hat{\theta}_{\text{MLE}} = \frac{k}{n} = \frac{4}{4} = \mathbf{1.0}$$

Step 2 — MAP (Mode of $\text{Beta}(3+4, 3+0) = \text{Beta}(7, 3)$):

- $\text{Beta}(3, 3)$ adds $\alpha - 1 = 2$ imaginary heads and $\beta - 1 = 2$ imaginary tails (4 imaginary flips total):

$$\hat{\theta}_{\text{MAP}} = \frac{4 + (3-1)}{4 + (3+3-2)} = \frac{6}{8} = \mathbf{0.75}$$

Step 3 — Posterior Mean (Expected Value of $\text{Beta}(7, 3)$):

$$\mathbb{E}[\theta \mid D] = \frac{4+3}{4+3+3} = \frac{7}{10} = \mathbf{0.70}$$

🎯 Exam Takeaway & Fast Shortcuts

- Don't confuse **MAP (Posterior Mode)** $\frac{k+\alpha-1}{n+\alpha+\beta-2}$ with **Posterior Mean** $\frac{k+\alpha}{n+\alpha+\beta}$! When $\alpha = \beta = 1$ ($\text{Uniform}(0, 1)$ flat prior), $\hat{\theta}_{\text{MAP}} = \frac{k}{n} = \hat{\theta}_{\text{MLE}}$, whereas the Posterior Mean is Laplace's Rule of Succession $\frac{k+1}{n+2}$.

3.5 🌟 [Core — Bread & Butter] Information Theory: Entropy, Cross-Entropy, KL Divergence & Mutual Information

💡 Intuition (Plain English First)

Information Theory answers a simple question: **How surprised are you when an event x happens, and how many bits does it take to transmit that event?**

- If an event is 100% certain ($P(x) = 1$), your surprise is $\log \frac{1}{1} = 0$.
- If an event is rare ($P(x) = \frac{1}{8}$), your surprise is $\log_2 \frac{1}{1/8} = 3$ bits.

From this single definition of surprise $\log \frac{1}{P(x)} = -\log P(x)$, four foundational ML quantities emerge:

1. **Shannon Entropy $H(P)$:** Your **average surprise** under the true distribution P: $H(P) = \mathbb{E}_{x \sim P}[-\log P(x)]$.
   - A deterministic coin ($P(\text{Heads}) = 1$) has $H(P) = 0$ bits (zero uncertainty).
   - A fair K-sided die ($P(x) = 1/K$) has the **maximum possible entropy** $H(P) = \log K$ (maximum uncertainty).
2. **Cross-Entropy $H(P, Q)$:** Suppose the real world follows distribution P, but your neural network predicts distribution Q. Cross-Entropy is your **average surprise when using your model Q's code to compress real-world events from P**:

   $$H(P, Q) = \mathbb{E}_{x \sim P}[-\log Q(x)] = -\sum_{x} P(x) \log Q(x)$$

3. **KL Divergence $D_{\text{KL}}(P \parallel Q)$:** The **extra wasted bits (penalty)** you pay because your model Q does not match the true distribution P:

   $$D_{\text{KL}}(P \parallel Q) = \underbrace{H(P, Q)}_{\text{Bits using model } Q} - \underbrace{H(P)}_{\text{Ideal bits using true } P} = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} \geq 0$$

   *(Why do we minimize Cross-Entropy $H(P, Q)$ when training a neural network instead of $D_{\text{KL}}(P \parallel Q)$? Because the true data entropy $H(P)$ does not depend on our model weights θ, so $\nabla_\theta H(P, Q) = \nabla_\theta D_{\text{KL}}(P \parallel Q)$—minimizing Cross-Entropy is **100% identical** to minimizing KL Divergence!)*
4. **Mutual Information $I(X; Y)$:** How much your uncertainty about X **drops** once someone tells you Y:

   $$I(X; Y) = H(X) - H(X \mid Y)$$

   Unlike linear correlation $\rho(X, Y)$, Mutual Information captures **every possible non-linear dependency**: $I(X; Y) = 0$ **if and only if X and Y are strictly independent**!

📐 The Math & Every Symbol Explained

1. Core Information Theory Identities

- **Shannon Entropy:** $H(P) = -\sum_x P(x) \log P(x) \in [0, \log K]$ (by convention, $0 \log 0 = 0$).
- **Cross-Entropy Decomposition:**

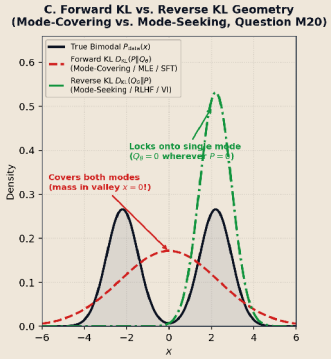
  $$H(P, Q) = H(P) + D_{\text{KL}}(P \parallel Q) \geq H(P)$$

- **Gibbs' Inequality:** $D_{\text{KL}}(P \parallel Q) \geq 0$, with equality $D_{\text{KL}}(P \parallel Q) = 0 \iff P = Q$.
- **KL Divergence Is NOT a True Distance Metric:**
  1. It is **asymmetric:** $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$.
  2. It violates the triangle inequality.

2. Forward KL $D_{\mathrm{KL}}(P \parallel Q)$ vs. Reverse KL $D_{\mathrm{KL}}(Q \parallel P)$ (Crucial for Generative AI & RLHF!)

Suppose the true distribution $P(x)$ is bimodal (has two separate peaks) and we fit a single-peak Gaussian model $Q_\theta(x)$:

- **Forward KL $D_{\mathrm{KL}}(P \parallel Q_\theta) = \sum_x P(x) \log \frac{P(x)}{Q_\theta(x)}$ (Mode-Covering / Mean-Seeking):** Look at where $P(x) > 0$: if our model sets $Q_\theta(x) \to 0$ anywhere that $P(x) > 0$, then $\log \frac{P(x)}{0} \to +\infty$! Thus Q_θ is terrified of missing any peak of P and stretches wide to cover **all modes** of P (even putting probability mass in the empty valley between the peaks). *Used in standard MLE and Supervised Fine-Tuning (SFT).*
- **Reverse KL $D_{\mathrm{KL}}(Q_\theta \parallel P) = \sum_x Q_\theta(x) \log \frac{Q_\theta(x)}{P(x)}$ (Mode-Seeking / Zero-Forcing):** Look at where $P(x) = 0$ (the empty valley between the peaks): if $Q_\theta(x) > 0$ there, $\log \frac{Q_\theta(x)}{0} \to +\infty$! So Q_θ is forced to be 0 wherever $P(x) = 0$, and happily locks onto **one single sharp peak (mode)** of P while ignoring the other peak. *Used in Variational Inference (VAE) and RLHF/PPO KL penalty.*

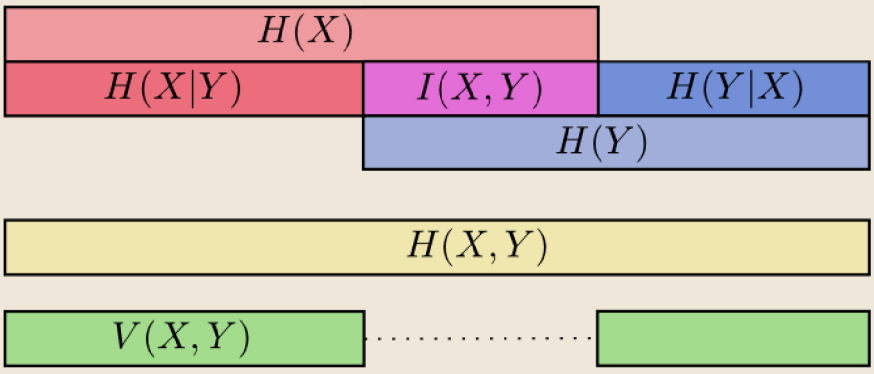


Forward KL (Mode-Covering) vs. Reverse KL (Mode-Seeking)

3. Mutual Information & Joint Entropy Venn Diagram Identities

$$H(X,Y) = H(X) + H(Y \mid X) = H(Y) + H(X \mid Y)$$

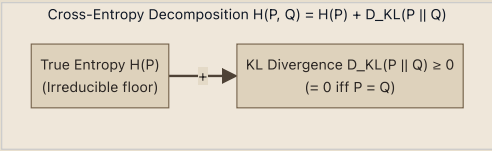
$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y) = D_{\mathrm{KL}}\big(P(X,Y) \parallel P(X)P(Y$$

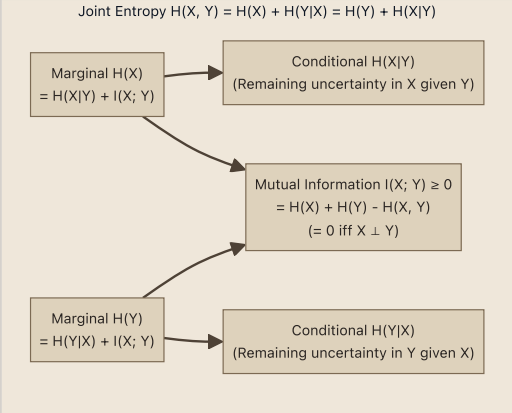


Visual Information Decomposition: H(X), H(Y), Conditional Entropies H(X|Y), H(Y|X), Mutual Information I(X;Y) & Joint Entropy H(X,Y) (Chris Olah)

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Continuous Differential Entropy Can Be Negative!**

For discrete random variables, $0 \leq P(x) \leq 1 \implies -\log P(x) \geq 0 \implies H(X) \geq 0$ always. However, for a **continuous** random variable with density $f(x)$, the PDF $f(x)$ can exceed 1 on a narrow interval! For example, if $X \sim \text{Uniform}(0, \frac{1}{2})$, then $f(x) = 2$ on $[0, \frac{1}{2}]$, so its **differential entropy** is $h(X) = -\int_0^{1/2} 2 \log_2(2)\, dx = -1 < 0$! (By contrast, $D_{\text{KL}}(P \parallel Q) \geq 0$ and $I(X;Y) \geq 0$ remain strictly non-negative for both discrete and continuous variables).





🔢 Step-by-Step Toy Example

Problem: Let the true distribution over 2 classes be $P = [0.5, 0.5]$ and a model's predicted distribution be $Q = [0.25, 0.75]$. Using $\log_2$ (bits), compute:

1. True Entropy $H(P)$
2. Cross-Entropy $H(P, Q)$
3. KL Divergence $D_{\text{KL}}(P \parallel Q)$

Step 1 — Compute True Entropy $H(P)$:

$$H(P) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = -0.5(-1) - 0.5(-1) = \mathbf{1} \text{ bit}$$

Step 2 — Compute Cross-Entropy $H(P, Q)$:

$$H(P,Q) = -0.5 \log_2 \left(\frac{1}{4}\right) - 0.5 \log_2 \left(\frac{3}{4}\right) = -0.5(-2) - 0.5(\log_2 3 - 2) = 2 - 0.5 \log_2 3 \approx 2 - 0.5(1.585) = \mathbf{1.}$$

Step 3 — Compute KL Divergence $D_{\text{KL}}(P \parallel Q) = H(P, Q) - H(P)$:

$$D_{\text{KL}}(P \parallel Q) = 1.2075 - 1.0 = \mathbf{0.2075} \text{ bits} > 0 \checkmark$$

🎯 Exam Takeaway & Fast Shortcuts

- **The One-Hot Target Shortcut:** In standard classification where the true label $P = y$ is a **one-hot vector** (e.g., $P = [0, 1, 0]$), the true distribution is 100% deterministic, so its entropy is $H(P) = 0$! Therefore, for one-hot labels, **Cross-Entropy equals KL Divergence exactly:** $H(P, Q) = D_{\text{KL}}(P \parallel Q) = -\log Q(y_{\text{true}})$.